

AI Serving Framework — Project Prompt Pack

Seven production-shaped projects across GPU server, mobile and IoT

Contents

Project Prompt Pack	3
The failure these prompts are written against	3
How to use these	3
The pack	4
What every prompt enforces	5
One rule worth reading before you start	5
Building the PDFs	5
Shared Requirements	6
1. What every project must produce	6
2. Frontend: three tabs, mandatory	6
3. Architecture: the component chain	9
4. Optimisation: substitute honestly	9
5. Compression is mandatory, and it must be gated	10
5a. RLHF: the improvement loop is mandatory	10
5b. Nothing is promoted without beating what it replaces	12
6. Security, audit and compliance	13
7. Monitoring: measure the right signal	13
8. Engineering standards	13
8a. The node data model, and the README section that closes the argument	14
9. Deliverables checklist	15
Project 01 — Interactive Voice Agent (TensorRT-LLM)	16
0. Numbered build plan	16
1. Brief	16
2. Why TensorRT-LLM for this workload	17
3. Model and compression	17
4. Architecture	17
5. Frontend	18
6. Architecture graph: node inventory	19
7. The improvement loop for this project	20
7a. LoRA adapters: where they earn their place here	20
8. The release gate for this project	20
9. Monitoring and KPIs	21
10. Security, audit and compliance	21
11. Deployment	22
12. Deliverables	22
13. Out of scope	22
Project 02 — Document & Image Analysis (SGLang)	23
0. Numbered build plan	23
1. Brief	23
2. Why SGLang for this workload	24
3. Model and compression	24
4. Architecture	24
5. Frontend	25

6. Architecture graph: node inventory	25
7. The improvement loop for this project	27
7a. LoRA adapters: where they earn their place here	27
8. The release gate for this project	27
9. Monitoring and KPIs	28
10. Security, audit and compliance	29
11. Deployment	29
12. Deliverables	29
13. Out of scope	29
Project 03 — Offline Field Assistant (llama.cpp)	30
0. Numbered build plan	30
1. Brief	30
2. Why llama.cpp for this workload	31
3. Model and compression	31
4. Architecture	31
5. Frontend	32
6. Architecture graph: node inventory	33
7. The improvement loop for this project	34
7a. LoRA adapters: where they earn their place here	34
8. The release gate for this project	35
9. Monitoring and KPIs	35
10. Security, audit and compliance	36
11. Deployment	36
12. Deliverables	36
13. Out of scope	36
Project 04 — On-Device Face Verification (ExecuTorch)	37
0. Numbered build plan	37
1. Scope, stated first	37
2. Why ExecuTorch for this workload	38
3. There is no KV cache here — and that is the point	38
4. Models and compression	38
5. Architecture	39
6. Frontend	40
7. Architecture graph: node inventory	40
8. The improvement loop for this project	41
8a. LoRA adapters: where they earn their place here	41
9. The release gate for this project	42
10. Monitoring and KPIs	42
11. Security, audit and compliance	43
12. Deployment	43
13. Deliverables	43
14. Out of scope	44
Project 05 — Workplace Safety Tracking (ONNX Runtime)	45
0. Numbered build plan	45
1. Brief	45
2. Why ONNX Runtime for this workload	46
3. No KV cache here — the substitution	46
4. Models and compression	46
5. Architecture	47
6. Frontend	48
7. Architecture graph: node inventory	48
8. The improvement loop for this project	49
8a. LoRA adapters: where they earn their place here	49
9. The release gate for this project	50
10. Monitoring and KPIs	51
11. Security, audit and compliance	51
12. Deployment	51

13. Deliverables	52
14. Out of scope	52
Project 06 — Drone Infrastructure Inspection (LiteRT)	53
0. Numbered build plan	53
1. Brief	53
2. Why LiteRT for this workload	54
3. No KV cache here — the substitution	54
4. Models and compression	54
5. Architecture	55
6. Frontend	56
7. Architecture graph: node inventory	56
8. The improvement loop for this project	57
8a. LoRA adapters: where they earn their place here	58
9. The release gate for this project	58
10. Monitoring and KPIs	59
11. Security, audit and compliance	60
12. Deployment	60
13. Deliverables	60
14. Out of scope	60

Project Prompt Pack

The failure these prompts are written against

Most AI projects do not fail because the model was wrong. They fail in four predictable ways, and every requirement in this pack exists to close one of them:

A demo that never becomes a system. Something works in a notebook, everyone is impressed, and nothing reaches a user. The gap is authentication, scaling, observability, rollout and rollback — none of which is interesting, all of which is the actual project.

A per-token bill that grows with success. Hosted APIs price the thing you were trying to increase. Costs scale with adoption, the model is deprecated on someone else’s schedule, and the data leaves the building. Every project here self-hosts and states a fixed cost in the units the client actually pays.

Collapse under load, invisible on the dashboard. Inference saturates the accelerator, not the CPU, so a CPU-target autoscaler never fires and every infrastructure panel stays green while the product times out. Every project scales on the signal that genuinely saturates, and separates “the hardware is healthy” from “the system is useful” into two dashboards that are allowed to disagree.

Nobody can say what it was told, or who changed it. The prompt, the thresholds and the policies are the product’s behaviour, and they usually live in a branch, a config file and someone’s memory. A customer gets a wrong answer and no one can reconstruct what was live at the time. Every project puts that surface in front of a non-engineer, versions it immutably, and records every change in a tamper-evident log.

The frontend requirements are not decoration for the same reason. A buyer cannot evaluate a system they cannot see, and an internal team cannot align on one they cannot point at. The Architecture tab exists so that “why does this cost what it costs” has a visual answer, and the guided tour exists so the same answer is given the same way twice.

Copy-paste briefs for building production-shaped AI serving projects against seven different runtimes. Each is written to be pasted whole into a fresh repository as the opening specification.

The vLLM customer support assistant in this repository is the reference implementation. Where a prompt is ambiguous, that codebase is the tie-breaker — it is the worked example of the standard the others are held to.

How to use these

1. Create a new, empty repository for the project.
2. Paste `00-shared-requirements.md` and the project prompt together as the brief.

3. Feed **§0, the numbered build plan**, into Claude Code’s plan mode. Every project prompt opens with one: twelve numbered steps, each independently reviewable, with the first four being what has to land before the demo is filmable.
4. Expect to answer the same four questions each project needs settled early: cloud target, compression target, scope of the configuration surface, and what hardware is available for development.

The prompts deliberately state acceptance criteria rather than implementation detail. They describe what must be true when it is finished, not how to type it.

What each project prompt now contains

Beyond the brief and the framework rationale, every prompt carries three sections that make the UI reproducible rather than merely described:

- **Architecture graph: node inventory** — the actual stage cards for that domain, in pipeline order, with what each expands into. This is the content for the 3D graph; the *mechanics* live in `00-shared-requirements.md §2`.
- **The improvement loop for this project** — which feedback signals are cheap to collect in that modality, and the domain rules that are expensive to get wrong (never train on captured faces; judge voice on the transcript; store the tile, not the frame).
- **The release gate for this project** — the two benchmark axes and the specific thresholds that matter, including where `lm_eval` and GuideLLM do not apply and what replaces them.

The shared requirements are unusually prescriptive about the graph

`00-shared-requirements.md §2` reads as a long list of rules because each one was learned by getting it wrong on camera first: pinning two axes makes it look 2D, expanding in place buries the thing you are talking about, perspective-scaled icons are nine pixels wide, and re-framing the camera on every simulation settle makes dragging look broken. Follow it rather than re-deriving it.

The pack

#	Framework	Class	Project
—	vLLM	GPU server	Customer support assistant (built — this repo)
01	TensorRT-LLM	GPU server	Interactive voice agent, sub-800 ms turn latency
02	SGLang	GPU server	Document and image analysis, structured extraction
03	llama.cpp	Mobile	Offline field assistant, fully on-device
04	ExecuTorch	Mobile	On-device face verification, consented 1:1
05	ONNX Runtime	IoT / edge	Workplace safety tracking, zone intrusion
06	LiteRT	IoT / edge	Drone infrastructure inspection

Each project was matched to the framework whose actual strength it exercises, rather than distributed arbitrarily:

- **TensorRT-LLM** □ **voice**. Time to first token sits directly in the turn-latency budget, and in-flight batching plus request cancellation are what make barge-in work. This is the workload where TensorRT-LLM’s compilation cost is repaid.
- **SGLang** □ **documents**. RadixAttention reuses the KV cache across branching shared prefixes, which is precisely what thousands of same-type documents produce. Constrained decoding removes invalid

JSON as a failure class outright.

- **llama.cpp** $\square$ **offline mobile**. Minimal dependencies, GGUF quantization and mmap loading are what let a model run at all on a phone with no network.
- **ExecuTorch** $\square$ **face verification**. Direct PyTorch export, NPU delegation, and a small runtime, for a fixed vision pipeline that may need retraining on customer data.
- **ONNX Runtime** $\square$ **edge tracking**. One .onnx artifact across a heterogeneous device fleet via execution providers, which is the real operational problem at edge scale.
- **LiteRT** $\square$ **drone**. Best inference-per-watt on EdgeTPU-class hardware, plus pruning and quantization-aware training in one toolkit, for a workload where the compute budget is capped by battery physics.

What every prompt enforces

Set out in full in `00-shared-requirements.md`:

- **Three frontend tabs** — Product, Architecture, Monitoring
- **A 3D click-to-focus architecture graph** where every component explains what it does, why it exists, what it saves the client, and what the user feels
- **The full component chain** — auth, authorization, retrieval, skills, inference, guardrails, stateless logging, audit, monitoring, staging/production, orchestration
- **Compression with a quality gate that can fail the build**
- **Hash-chained audit logging** with HIPAA / SOC 2 / GDPR control mapping
- **Prometheus and Grafana** covering latency, throughput and 4xx
- **Measured numbers, never estimated ones**

One rule worth reading before you start

The shared requirements carry a substitution rule, because applying the vLLM checklist unmodified to a vision model produces nonsense:

KV cache, continuous batching and prefix caching exist only where there is autoregressive token generation. They apply to TensorRT-LLM, SGLang and llama.cpp. They do not apply to ExecuTorch face verification, ONNX Runtime object tracking, or LiteRT drone imagery — those run one forward pass per input and have no KV state to cache.

For the vision projects the equivalent story is INT8 quantization, operator fusion, NPU delegation, input resolution tuning and pruning. Each of those prompts names its substitute explicitly and asks for it to be shown on the architecture node in the same position the KV cache would occupy. The same applies to Kubernetes: mobile and IoT projects use a model registry with staged rollout rings, which is the honest analogue of staging $\square$ production, not a lesser version of it.

Being right is worth more than being uniform. A client with domain knowledge will notice a fabricated KV cache immediately, and it costs more credibility than the consistency was worth.

Building the PDFs

```
./scripts/build-prompt-pdfs.sh
```

Writes one PDF per prompt to `prompts/pdf/`, plus a combined `all-projects.pdf`. Requires pandoc and a LaTeX engine (xelatex).

Copy from the markdown, not the PDF. The PDFs are for reading and sending to people; the .md files are the source you paste into a new repository. LaTeX sets common ligatures, so text copied out of a PDF can contain `f l` (U+FB02) where you expect `f l` — harmless when read, quietly wrong when pasted into a spec.

Shared Requirements

Every project prompt in this pack assumes this document. Paste it into the target repo alongside the project-specific prompt, or paste both together as one brief.

The reference implementation of everything below is the **vLLM customer support assistant**. Where a requirement is ambiguous, that repo is the tie-breaker.

1. What every project must produce

A **demonstrable** system, not a notebook. The test is whether a prospective client can be shown a five-minute video and understand what it does, how it works, and why each component earns its cost.

That means three things must exist and work together:

1. A working product surface someone can actually use.
2. An architecture view that explains the system to a non-engineer.
3. A monitoring view that proves it is observable, secure and auditable.

2. Frontend: three tabs, mandatory

React + Next.js (App Router), TypeScript strict. Three top-level tabs.

Tab 1 — Product

The working demo. Whatever the project does — chat, voice, detection, recognition — this is where it is exercised live. It must be usable by someone who has never seen it, with no CLI and no configuration file editing.

Include a configuration surface where it makes sense for the domain (company name, policies, thresholds, tracked classes, enrolled identities). Configuration changes must visibly change behaviour, and the change must be inspectable before it is applied.

Tab 2 — Architecture Design

An interactive **3D force-directed graph** (`react-force-graph-3d`, which wraps `3d-force-graph`), built as an **expandable pipeline**, not a flat diagram.

This section is unusually prescriptive because the obvious implementations were each tried and each failed on camera. What follows is the design that survived.

The shape: a pipeline of expandable stage cards The graph opens showing **7–9 top-level stage cards** in the order a request actually travels, left to right. Each stage expands on click into its parts.

- **Pin one axis only.** Give each top-level stage a `flowOrder` and pin its `x` to $(\text{flowOrder} - (\text{count} - 1) / 2) * \text{SPACING}$. Leave `y` and `z` free.
 - Pinning two axes produces a flowchart drawn in WebGL. The first version of this pinned `x` and `y`; the client's words were *"it is kinda 2d"*.
 - Pinning none produces a sphere that is not architecture.
- **Child nodes are never pinned.** They cluster around their parent under the link force, which is what makes an expanded card read as *contained*.
- **The root is logical, not a node.** Parent every stage to a `ROOT_ID` that has no node of its own. Rendering a root produces a hub every stage hangs off — exactly the shape a pipeline is meant to replace, and it leaves the first stage visibly dangling.

Interaction: click to isolate, click again to return

- **Clicking a stage shows only that stage and its contents.** Everything else leaves the scene. Expanding in place leaves the thing being discussed competing with thirty other nodes — on a recording it becomes the *least* prominent element on screen.

- **Clicking the same node again restores the full network**, collapsed. The way back must be the thing you just clicked; a presenter should not have to hunt the toolbar mid-sentence. Provide Esc and a visible “Back to full network” button as well, because a view that hides everything needs a stated exit.
- **Nodes are draggable and stay where dropped** (set fx/fy/fz on drag end). The force layout is good at untangling and has no idea which arrangement makes the argument clearest.
- **The user owns the camera.** Auto-framing must stand down the moment someone drags. Implement it as: a pointer press that *moves* more than a few pixels claims the camera; a press that does not is still a click. Re-framing on every simulation settle is the single worst bug in this component — the graph springs back a beat after every drag and reads as “panning is broken”.
- **Reset squares up before fitting.** zoomToFit only changes distance, so after a rotation it re-frames the pipeline while leaving it standing on end. Put the camera back on the +Z axis first, then fit.
- **Fit correctly.** If you tighten past zoomToFit, scale the camera position toward the **controls target**, not the world origin — the graph’s bounding box is not centred on the origin, and scaling toward the origin swings the view direction and slides the graph into a corner.
- **Refit when the simulation stops**, not on a timer. A timed fit frames a layout that no longer exists a second later.

Node visuals: icons, and constant screen size

- **Every node carries a glyph** — a database cylinder, a shield, a chip, a person, a gateway. A client should recognise the database before reading a word. Shape survives video compression; 11px type does not.
- **Draw glyphs on a canvas; do not fetch logo files.** The Architecture tab has to render offline and in CI, and a missing logo mid-demo is worse than a simplified one. Brand SVGs are also trademarked artwork — a recognisable glyph in the product’s own colour identifies the technology without redistributing someone’s mark.
- **Nodes and labels must be constant size on screen** (sizeAttenuation: false on the sprite material). A pipeline spans thousands of world units while a node is ~10 across, so a perspective-scaled node is under ten pixels at any framing that fits the whole diagram. The cost is losing size-as-depth-cue; the links and the flow axis already carry that.
- **Hang labels off the sprite’s center, not a world-space offset.** A world offset shrinks toward nothing as the camera pulls back, and the label ends up sitting on top of the node it names.
- **Show that a card opens.** A collapsed parent needs a dashed ring or shell and a +N inside count. Without it, expandability is a secret.

Links

- **Roll every request-path edge up to the nearest *visible* ancestor.** Without this the collapsed view has no flow at all: every real edge runs between leaf nodes hidden inside collapsed cards, so the diagram shows containment and nothing else. Aggregating gives Gateway → Context because authz → retriever exists underneath it, and the edges resolve to finer granularity as cards open.
- **Copy link objects before handing them to the graph.** The library rewrites source/target from ids to node objects *in place*; passing your module-level constant means the second render is comparing objects against a set of ids and silently dropping every edge.
- **Give each relationship kind its own colour** — containment, request path, context being pulled in, data being written, observation, improvement feedback.
- **Set two rest lengths.** Containment edges short, so a card reads as one cluster. Flow edges *longer than the pinned spacing*, so the only way to satisfy them is to separate in y and z — that is what lifts the pipeline off the line into a ribbon with real depth, and what stops it settling flat.

The detail panel HTML, never 3D text. Prose rendered as WebGL is unreadable in a compressed video, and the cost figures are the part that has to be read. The scene carries structure; the panel carries meaning.

Each node’s panel carries four fields, all mandatory:

Field	Answers
What it does	One line, plain language
Why it is used	Why this component exists at all — what breaks without it

Field	Answers
Client benefit (cost)	What it saves, with a number wherever one has been measured
User benefit (quality)	What the end user actually feels

Plus a **“Say this”** line: the sentence the presenter speaks when the node is on screen. It is a script prompt that also reads as a summary to the viewer.

Where a component can be *shown* rather than described — the product’s own UI, a sample detection, a rendered result — put a small worked example in its panel. Build it from the real components rather than embedding a screenshot: it must render with the backend off, it stays legible at any zoom, and it cannot drift out of date with the design system.

The guided tour An ordered walk through every significant node, with narration, arrow-key navigation and autopilot. **This is what gets recorded**, so it must:

- Open on the user’s problem, not on the technology.
- Follow the pipeline in order and never double back.
- Expand every ancestor of a stop before selecting it, or the camera flies at a node still folded inside a collapsed parent and lands on nothing.
- Cover every component that carries the commercial argument.
- Include the honest stops — the technique that is *not* enabled, and why.

Non-negotiable **The Architecture tab must work with the backend switched off.** It is static data. A demo must never fail because a GPU is cold. Prove it with an end-to-end test that runs with no backend, rather than asserting it in a README.

Tab 3 — Monitoring

Four sections: **Quality, Performance, Security, Audit.**

- **Quality** — task-appropriate accuracy, and the rate at which the system correctly declines rather than guessing.
- **Performance** — latency percentiles (p50/p95/p99), throughput, hardware utilisation, and the framework’s headline optimisation in action.
- **Security** — attacks detected and blocked, PII handling, authentication and authorisation failures, 4xx/5xx rates.
- **Audit** — an append-only, tamper-evident event log with compliance mapping.

Live data when available, seeded demo data otherwise, with a visible badge. The API response carries a source field ("live" or "demo") and the UI renders the badge from it. Never present synthetic numbers as though they were live.

Frontend quality bar

The interface is being shown to buyers. Enforceable constraints, not aspirations:

- One neutral colour ramp plus **one** accent. Accent is reserved for focus, links and selection. Primary actions are near-black on light and near-white on dark — colour on a button should mean something specific, like destruction.
- No gradient hero sections, no glassmorphism, no purple/blue gradient accents, no emoji used as icons.
- Every colour comes from a design token. No ad-hoc hex values in components.
- WCAG AA contrast, visible focus-visible rings, prefers-reduced-motion honoured, full keyboard operation, correct ARIA on custom controls.
- Light and dark themes, following the OS by default with an explicit override.

3. Architecture: the component chain

Every project must include every one of these, adapted to its device class. If a component genuinely does not apply, say so explicitly in the node's detail panel and name the substitute — do not silently drop it.

One exception, and it is the one people get wrong: RAG. Retrieval-augmented generation belongs in a system that answers questions from a body of documents. It does **not** belong in a face-recognition system, an object tracker or a drone-imagery pipeline. Those have a *context assembly* step — an enrolment index, a frame buffer, a tile with its geo-reference — and that is a different thing wearing the same slot in the diagram.

Do not put a “RAG retrieval” node in a vision project. There is no corpus, there is no query embedding, and a client who works in the domain will spot it immediately and start discounting everything else on the page. Name the real component instead:

Project	The context-assembly step is actually
Voice agent, document analysis, offline assistant Face recognition	RAG — genuine retrieval over a document corpus Enrolment index lookup — cosine similarity over stored templates
Object tracking	Frame pipeline + track state — the previous frames and open tracks
Drone imagery	Tiling + geo-reference — the tile the model sees and where it is in the world

The same discipline as section 4's KV-cache rule: uniformity across the pack is worth nothing next to being right about the domain.

User action (Product tab)

- Client application
- Edge: ingress, TLS, rate limiting
- Authentication (who are you)
- Authorization (what may you do)
- Request analysis / intent
- Context assembly — RAG *only if* there is a document corpus to retrieve from; otherwise the real equivalent (enrolment index, track state, tile + geo-reference). See the note below.
- Skills / tools / business logic
- INFERENCE ENGINE ← the framework this project is about
 - + its headline optimisation (see section 4)
 - + its compression strategy (quantization, pruning, distillation)
- Post-processing, guardrails, confidence gating
- Stateless structured logging (so instances scale horizontally)
- Audit log: append-only, hash-chained, HIPAA / SOC 2 / GDPR mapped
- Monitoring: latency, throughput, 4xx/5xx — Prometheus + Grafana
- Staging and production environments
- Orchestration and scaling
- IMPROVEMENT LOOP ← feedback capture → preference data → training → gate
(section 5a — the only edges that flow back upstream)

Stateless is a requirement, not a preference. No inference or application node may hold session state in memory. State belongs in the database or the client. This is what allows horizontal scaling, and it is worth calling out on the node.

4. Optimisation: substitute honestly

This is the rule most likely to be got wrong, so it is stated bluntly.

KV cache, continuous batching and prefix caching exist only where there is autoregressive token generation.

Project	KV cache applies?	Headline optimisation to demonstrate instead
TensorRT-LLM voice agent	Yes	Paged KV, in-flight batching, fused kernels, FP8
SGLang document analysis	Yes	RadixAttention prefix reuse, structured decoding
llama.cpp offline assistant	Yes	GGUF quantization, mmap load, Metal/NNAPI offload
ExecuTorch face recognition	No	INT8 PTQ, XNNPACK/Core ML delegation, operator fusion
ONNX Runtime object tracking	No	INT8 quantization, graph fusion, execution providers
LiteRT drone imagery	No	INT8 full-integer quant, NPU/EdgeTPU delegation, pruning

For the three vision projects, **do not fabricate a KV cache**. A detection model runs one forward pass per frame — there is no autoregressive state to cache. The node in the architecture graph should be labelled with the real optimisation and its detail panel should say plainly why KV caching does not apply here. A client who knows the domain will notice, and being right is worth more than being uniform.

Likewise for orchestration:

Device class	“Staging → production” means
GPU server	Kubernetes, staged rollout, HPA on the right signal
Mobile	Model registry + staged OTA rollout rings, phased release
IoT / edge	Fleet management, canary devices, signed model artifacts, rollback

A phone does not run Kubernetes. The requirement is a **controlled path from staging to production with a rollback**, not the specific tool.

5. Compression is mandatory, and it must be gated

Every project compresses its model, and every project proves the compression did not break it.

- State the technique, the target hardware, and the memory before and after.
- Explain **what the freed memory buys** — that is the client-facing point. On a GPU server, smaller weights become KV cache, which becomes concurrent users. On a phone, it becomes the difference between running and not running.
- Ship a **quality gate** that fails the build when accuracy drops past a threshold. Compare against the uncompressed baseline. Make the scoring deterministic — a gate that blocks releases must give the same answer twice.
- Include a small task-specific behavioural suite alongside any academic benchmark. General benchmarks catch capability damage; only a domain suite catches the model becoming useless at the actual job.

5a. RLHF: the improvement loop is mandatory

Everything above describes a system that stays the same. Ship the part that compounds: **reinforcement learning from human feedback** — human judgements about outputs become the training data for the next version of the model.

“RLHF” here means the modern, practical form of it: collect preferences, train with DPO on a LoRA adapter, gate the result. Not a reward model and a PPO loop — that is far more machinery for the same signal, and on a few thousand domain preferences the reward model would be the weakest part of the pipeline. Where

a project has no generative model to fine-tune (the vision projects), the same loop runs with the training step swapped for retraining or threshold policy; say which and do not pretend otherwise.

This is also the strongest commercial argument in the deck. It is the difference between “the accuracy you buy is what you get” and “the accuracy you buy is a floor” — and unlike a hosted API, the judgements your client’s team makes stay theirs.

Collection is automatic

Build these into the Product tab, not into a separate admin tool:

Signal	What it is	Why it is worth collecting
Rating	Thumbs up / down on one output	Cheap and plentiful; weak on its own, because people disagree about what “good” means in the abstract
Preference	“This output is better than that one” for the same input	Far stronger. A comparison sidesteps the calibration problem entirely, and it is the native input format for DPO and for reward-model training
Correction / comment	Free text, or the corrected label	Not directly trainable, but the only signal that says <i>why</i> , and the one that tells you which failure to go fix

Rules that are easy to get wrong and expensive to fix later:

- **Ask at the moment of judgement.** Under the output, while the person still has an opinion. Ask an hour later and you collect recall, not judgement.
- **Use ± 1 , not a 1–5 scale.** Five-point scales collapse to their extremes and the middle carries no usable signal, at the cost of a harder decision.
- **Comparisons must share their context.** Both candidates get the identical retrieved documents / identical input frame, so the judgement is about the model rather than about which side got luckier inputs.
- **Hide which variant is which until after the choice.** Knowing that A is “the current settings” is enough to bias the answer toward it, and a preference set with that bias baked in teaches the model to prefer what it already does.
- **Redact personal data on write.** This table is an export surface, and an export is the easiest way for personal data to leave the building — after which it is memorised by a model and cannot meaningfully be deleted.
- **Stamp the config/model version on every judgement.** A preference collected against an older prompt or an older model describes a system that no longer exists. Training on a mixture without knowing which is which drags a model toward its own past.
- **Store the triple denormalised** — (input, chosen, rejected). The exported example must be exactly what the annotator saw, and it must survive the source record being deleted under retention.
- **Mark what an export consumed,** so the export is resumable and the same judgement is not trained on twice and quietly over-weighted.

For the vision projects the same loop applies with the modality swapped: the judgement is “*this box / this identity / this classification is wrong*”, and a correction is a re-labelled frame. The pipeline shape does not change.

Training is *not* automatic, and say so

The loop stops at an export. A fine-tune that fires whenever enough judgements accumulate is a way to ship a regression that nobody read the data for.

- Export as JSON Lines in the format the trainer reads with no conversion step — a conversion step is exactly where a schema drifts out of sync with the trainer.

- Prefer **DPO over reward-model + PPO** for generative projects: it optimises directly on preference pairs with far less machinery to get wrong.
- Prefer a **LoRA adapter over a full fine-tune**: a few hundred MB, hours on one GPU, loadable at runtime, and rollback is unloading a file. The low-rank constraint is also a real guard against a narrow preference set making the model worse at everything else.
- **Know which half of the system LoRA lives in**. On a server runtime that supports adapters (vLLM, SGLang, TensorRT-LLM with the plugin compiled in) it is a *serving* technique too: one base model plus a small adapter per tenant, per document type or per vertical, selected at request time — which turns “one deployment per customer” into “one GPU, many specialisms” and is usually the strongest cost argument available. On an exported edge runtime (ExecuTorch, ONNX Runtime, LiteRT) it is a *training-only* technique: adapt, **merge, then export and quantize**. There is no adapter swapping on a frozen graph, and an architecture diagram that shows one in the serving path is wrong. Each project prompt states which case it is in.
- **Refuse to train below a floor** (a few hundred pairs). Below that you produce a model with strong opinions about a handful of inputs and worse behaviour everywhere else.
- Write a **training manifest** next to the artifact: base model, pair count, source file, hyperparameters. Six months later the only question that matters about a model in production is what it was trained on.

The Monitoring tab gets a fifth section

Add **Improvement** alongside Quality / Performance / Security / Audit: approval rate, judgements collected, head-to-head win rate of the challenger, and how many judgements are waiting for the next training run.

This is the one panel that is never seeded with demo data. Every other section falls back to synthetic numbers when the metrics backend is absent and says so with a badge. Feedback is a claim about what real people judged, and a plausible approval rate nobody gave is the one number on the dashboard that cannot be corrected by waiting for real traffic. An empty loop renders as empty.

5b. Nothing is promoted without beating what it replaces

Any artifact that changes the weights — quantization, pruning, or a fine-tune from collected feedback — faces the same gate, on **two axes that genuinely come apart**. A fine-tune can answer better *and* stream slower; a change that speeds generation up can cost accuracy. A gate that asks only one question ships the other regression.

Dimension	Question	Tool (generative)	Tool (vision / edge)
Quality	Is it still right?	lm_eval — instruction following, reasoning, plus a held-out domain suite	Task metrics on a held-out set: mAP, IoU, TAR@FAR, top-1
Performance	Is it still fast and efficient?	GuideLLM against a running server — p95 TTFT, inter-token latency, tokens/sec	On-device latency p95, memory peak, FPS, energy per inference

Rules:

- **Benchmark through the real serving path**, not an in-process engine. The number that matters is what a user experiences, including the scheduler, batching and transport. An in-process benchmark measures a configuration nobody runs.
- **Hold a realistic arrival rate**, not “as fast as possible”. Saturating the server measures peak throughput, which is not the operating point.
- **Relative bounds where a baseline exists** — “20% slower” means the same thing on any hardware, “+40ms” does not. **Absolute ceilings** where a number is unshippable regardless of history: a model that was already slow does not license the next one to be slower.
- **Absolute floors for behaviour**. A model that ignores the supplied policy, or that fails a liveness check, is unshippable whatever it scored on a benchmark.
- **Deterministic scoring**. A gate that blocks releases must give the same answer twice — so assertion-based, greedy decoding, no LLM judge adding its own variance to the signal being measured.

- **Either axis failing exits non-zero** and the artifact is not promoted.

Put both benchmarks in the architecture graph as their own nodes under the promotion gate. “We measure quality and speed before shipping” is a claim; two named tools with stated thresholds is evidence.

6. Security, audit and compliance

Security - Authentication and authorisation as separate, visible components. - Input validation and rate limiting at the edge. - Adversarial input detection appropriate to the modality — prompt injection for text and vision-language, and for the vision projects, at minimum an input sanity/liveness check where the domain calls for one. - Content received from a retrieval system or an uploaded document is **data, never instructions**. Treat and test it as an attack surface. - The inference endpoint is never internet-reachable. Only the application tier may reach it, enforced at the network layer. - No long-lived credentials. Workload identity or equivalent.

Audit - Append-only event log: actor, action, resource, outcome, timestamp. - Hash-chained so tampering is detectable. This is the property compliance buyers ask about, and it is cheap to implement. - Retention policy stated explicitly. - Personal data redacted **on the write path**, not filtered at read time. A filter someone can forget to apply is not a control.

Compliance mapping — for each audit event type, state which control it serves:

Regime	What must be demonstrable
GDPR	Lawful basis, data minimisation, erasure (Art. 17), special-category handling (Art. 9) where biometrics are involved
HIPAA	Access logging, minimum necessary, encryption at rest and in transit
SOC 2	Change management, access control, monitoring, incident evidence

Do not claim certification. Claim “*designed to support*” and show the evidence the control would need. Overclaiming here is a liability, not a selling point.

7. Monitoring: measure the right signal

Prometheus for metrics, Grafana for dashboards, structured JSON logs with a request ID echoed to the caller.

Two dashboards, because they answer different questions and can disagree:

1. **System** — latency, throughput, saturation, errors, hardware utilisation.
2. **Product** — is it actually useful? Task success rate, decline rate, confidence distribution, cost per successful outcome.

A system can be green on every infrastructure metric and useless. Only the product dashboard reveals that.

Autoscale on the signal that actually saturates. For GPU inference this is usually queue depth, not CPU — a saturated inference process is blocked on the accelerator while its CPU sits at 30%, so a CPU-target autoscaler never fires. Work out the real saturation signal for the project’s hardware and scale on that.

Alerts must be actionable. Every rule carries a runbook line saying what to check.

8. Engineering standards

- **Tests:** unit tests for the logic that carries the product’s promises. If a component’s correctness is the selling point, it has a test.
- **Types:** strict mode, no escape hatches without a comment explaining why.
- **CI:** lint, types, tests, and schema validation of any generated manifests.
- **Docker Compose** for a one-command local stack; the same images run in production.
- **Infrastructure as code** for anything cloud-side.

- **README** that a stranger can follow to a working demo.

Comments explain *why*, never *what*. The reasoning behind a non-obvious decision — why this layout, why this order, why this instance type — is the most valuable thing in the repo and the thing you will point at on camera. Dense, uncommented code looks efficient and reads as unconsidered.

8a. The node data model, and the README section that closes the argument

One typed record per node

The architecture graph is content, and content rots silently: a renamed id breaks a tour stop, a missing field renders an empty panel, and neither shows up until someone is recording. Keep it in **one typed file** with **tests**.

```
interface ArchNode {
  id: string;
  label: string;
  tier: TierId;           // which stage it belongs to – drives colour and filters
  parent?: string;       // hierarchy; ROOT_ID is logical and has no node
  flowOrder?: number;    // top-level stages only – the pinned axis
  sub?: string;          // one-line technical detail shown under the label
  icon?: IconKey;        // canvas-drawn glyph
  size?: number;

  what: string;           // the four mandatory rationale fields
  whyUsed: string;
  clientBenefit: string;
  userBenefit: string;

  metric?: { value: string; caption: string; estimated?: boolean };
  demoNote: string;      // the "Say this" line
}
```

Tests that are cheap and fail loudly — write all of them:

- Every node id is unique; every link connects two real nodes.
- Every node reaches the root; no parent cycles.
- Every node has all four rationale fields, non-trivially long.
- Every tour stop points at a real node, and the tour never goes backwards through the pipeline.
- Every icon key a node asks for exists — a typo indexes undefined and throws inside the render loop, taking the whole canvas down rather than one glyph.
- Every top-level stage has a glyph and a distinct flowOrder.
- The mandated component chain from section 3 is complete, asserted **by concept with a regex** rather than by id, so a rename cannot silently drop one.
- Anything not actually enabled is described as not enabled.

Be honest about what is not on

Include at least one node for a technique that is **built but deliberately disabled**, with the reason. In this reference implementation that is 2:4 pruning: the recipe ships, it is off by default, because one-shot sparsity costs instruction-following and that is the wrong thing for a support assistant to lose.

A client who checks one overstated claim discounts every other number in the deck. A stated limitation does the opposite — it makes the rest credible, and it is the single highest-leverage paragraph in the whole presentation.

The README must end with the tradeoff triangle

Every choice in these projects is a point on the same triangle: **cost, performance, accuracy**. Close the README with an image of it and a table where each row is one real decision, what it bought, and what it paid with. For example:

Decision	Bought	Paid with
INT4 instead of fp16	Weights that fit a commodity GPU	A small accuracy risk, which is why compression is gated rather than trusted
Pruning left off	Nothing — it ships disabled	A higher memory floor, because sparsity costs instruction-following
Escalating instead of guessing	Answers a user can trust	Deflection rate itself — every handoff is a task not resolved
Self-hosting rather than an API	Fixed cost that does not scale with success	Operational work: a model to run, update and roll out

Then say why the Monitoring tab exists: the triangle is not a one-time decision. Cost, latency and quality are measured continuously and side by side, so a change that improves one at the expense of another is visible rather than discovered later by a customer.

Where the framework supports it, add a second closing section on **portability** — which models, which accelerators, which deployment targets the same code reaches without a rewrite. Committing to a hosted API means committing to one vendor’s model, pricing and deprecation schedule simultaneously; keeping those as three separate reversible decisions is a commercial argument, not a technical one.

9. Deliverables checklist

Every project is done when all of these are true:

- ☐ Three tabs present and working
- ☐ 3D architecture graph: expandable pipeline, click-to-isolate, draggable nodes, icon glyphs, constant-size labels, and a guided tour
- ☐ Camera never fights the user — auto-framing stands down on drag
- ☐ Every component from section 3 present as a node with all four rationale fields plus a “Say this” line
- ☐ Architecture tab works with the backend off, proven by a test
- ☐ Feedback capture in the Product tab: rating, comparison, correction
- ☐ Preference export in the trainer’s own format, resumable
- ☐ Monitoring has a fifth Improvement section, never seeded
- ☐ Release gate measures quality *and* performance, and can fail on either
- ☐ README carries the cost/performance/accuracy tradeoff table
- ☐ Model compressed, with a quality gate that can fail
- ☐ Monitoring shows live-or-demo data with an honest badge
- ☐ Audit log append-only and hash-chained
- ☐ Prometheus metrics exposed; Grafana dashboards committed
- ☐ Staging → production path with rollback
- ☐ One-command local run
- ☐ CI green: lint, types, tests
- ☐ README a stranger can follow
- ☐ Measured performance numbers recorded — not estimated

Project 01 — Interactive Voice Agent (TensorRT-LLM)

Read `00-shared-requirements.md` first. Everything in it applies here.

0. Numbered build plan

Paste this section into Claude Code's plan mode to scope the work. Each step is independently reviewable; steps 1–4 must land before the demo is filmable.

1. **Repository skeleton.** Monorepo with `frontend/`, `backend/`, `model/`, `deploy/`, `observability/`. Strict TypeScript, typed Python, lint + types + tests in CI from the first commit.
2. **TensorRT-LLM on Triton serving layer.** Stand up the engine with the compression spec in §3–§4 below. Record weights and latency before and after, and state what the freed resource buys in the units the client pays for.
3. **Backend API.** The component chain in §5 as real services: authentication, authorization, adversarial-input detection, context assembly, inference, guardrails, structured logging, hash-chained audit log.
4. **Product tab.** The working demo, usable by someone who has never seen it, plus the configuration surface. Configuration changes must visibly change behaviour and be inspectable before they apply.
5. **Architecture tab.** The 3D expandable-pipeline graph, built to the spec in `00-shared-requirements.md` §2. Use the node inventory in §6 of this document as the content. Click-to-isolate, draggable nodes, canvas-drawn icon glyphs, constant-size labels, HTML detail panel with all four rationale fields plus a "Say this" line.
6. **Guided tour.** An ordered walk covering every node that carries the commercial argument, including the honest stop about what is *not* enabled. Arrow keys, autoplay, and it must run with the backend stopped.
7. **Monitoring tab.** Quality, Performance, Security, Audit, Improvement. Live data when available, seeded demo data otherwise, honest source badge — except Improvement, which is never seeded.
8. **Feedback capture.** Rating, side-by-side comparison and correction in the Product tab, per §7. Redact on write, stamp the model/config version, store the triple denormalised, mark what an export consumed.
9. **Export and training path.** JSON Lines in the trainer's own format, resumable. Training is operator-run, never automatic. Write a training manifest next to every artifact. Evaluate **LoRA adapters** per §7a — that section says where they belong in *this* project and where they do not.
10. **Release gate.** Both axes per §8 — quality and performance, each able to fail the build on its own. Wire it into CI.
11. **Deployment.** Containers, infrastructure as code, staging → production with a rollback, and autoscaling on the signal that actually saturates.
12. **README.** A stranger can reach a working demo. Ends with the cost/performance/accuracy tradeoff table, one row per real decision, and a portability section.

Acceptance is §12 of this document plus the shared deliverables checklist.

1. Brief

Build a **real-time voice agent** that a caller can interview, interrupt and argue with — a spoken support line, not a transcription demo. A user speaks into the browser, the agent answers in speech, and the exchange feels like a phone call rather than a walkie-talkie.

The engineering target that defines the project:

Under 800 ms from the user stopping speaking to the first audio of the reply, measured end to end at p95, with barge-in supported throughout.

That single number drives every decision below. Above roughly one second, people start talking over the agent because they assume it did not hear them; the conversation collapses. This is not a comfort target, it is the threshold at which the product works at all.

Target user: a company replacing an IVR phone tree with something that can actually answer questions, or adding a voice channel to an existing support desk.

2. Why TensorRT-LLM for this workload

Voice is the hardest latency case in LLM serving, and it is where TensorRT-LLM's specific advantages are decisive rather than marginal:

- **Time to first token is the product.** In a chat UI, TTFT is a comfort metric — text can stream in leisurely. In voice, TTFT gates when the first audio chunk can be synthesised, so it sits directly in the turn-latency budget. TensorRT-LLM's ahead-of-time kernel compilation, fused attention and layer fusion attack exactly this number.
- **In-flight (continuous) batching** keeps the GPU busy across concurrent calls without making any single caller wait for a batch to fill. A voice workload is bursty and latency-critical simultaneously — the worst case for static batching.
- **Paged KV cache** lets many concurrent calls share GPU memory efficiently. Conversations are long and mostly idle; without paging, each call would reserve its worst-case context and concurrency would collapse.
- **FP8 on Hopper/Ada** roughly halves weight memory and improves throughput, and on a latency-bound workload the memory freed becomes concurrency headroom.
- **Triton Inference Server integration** gives a production serving layer with request cancellation — which matters enormously here, because **barge-in means abandoning generation mid-stream**, constantly.

The honest counterpoint, state it in the README: TensorRT-LLM requires an engine-build step per model, per precision, per GPU architecture. You trade deployment flexibility for latency. For this workload that trade is correct; for a system that swaps models weekly it would not be.

3. Model and compression

Stage	Choice	Notes
ASR	distil-whisper/distil-large-v3 or Parakeet	Streaming, partial hypotheses
LLM	Llama-3.1-8B-Instruct or Qwen2.5-7B-Instruct	FP8 via TensorRT-LLM quantization toolkit
TTS	Piper, Kokoro, or XTTS-v2	Must support streaming chunk output

Non-negotiable TTS property: it must emit audio in chunks as text arrives, not after a complete sentence. A TTS that waits for the full reply adds its entire synthesis time to the turn budget and makes the target unreachable regardless of how fast the LLM is.

Compression spec - Build the TensorRT-LLM engine with FP8 weights and FP8 KV cache on Hopper/Ada (H100, L40S). Fall back to INT4 AWQ on Ampere. - Record weights before and after, and state what the freed memory buys — here it is concurrent calls per GPU, which is the unit the client is paying for. - Enable in-flight batching and paged KV in the engine config.

Quality gate - Academic: instruction-following benchmark, gated as a delta against the uncompressed baseline. - Domain suite: a held-out set of spoken support exchanges scored on whether the answer is correct, whether it stays inside supplied policy, and whether it correctly escalates when it does not know. - **Voice-specific:** measure word error rate through the full ASR → LLM → TTS loop, not on clean text. Compression damage shows up differently when the input is already noisy from ASR.

4. Architecture

The full chain from 00-shared-requirements.md §3, with these specifics:

Browser mic → WebRTC → Edge (TLS, TURN, rate limit)

- Authentication (OIDC) → Authorization (RBAC)
- VAD + endpointing ← turn boundary detection
- Streaming ASR ← partial hypotheses
- Intent analysis
- RAG retrieval (policies, account context)

- Skills / tools (lookup, booking, transfer)
- TensorRT-LLM on Triton
 - in-flight batching
 - paged KV cache (FP8)
 - FP8 quantized engine
- Guardrails + confidence gate
- Streaming TTS —WebRTC→ Browser speaker
- Stateless JSON logs
- Audit log (hash-chained, HIPAA/SOC2/GDPR)
- Prometheus → Grafana
- Staging / Production → Kubernetes + HPA

== RLHF / IMPROVEMENT LOOP ==

Turn rating + correction (operator review of transcripts)

- A/B: same question, two sampling settings, played back
- Preference dataset (transcript, chosen, rejected)
- Answer scoring → Grafana
- DP0 fine-tune (LoRA, operator-run, never automatic)
- Promotion gate: lm_eval (quality) + GuideLLM (performance)

==> back to the TensorRT-LLM engine

Design points worth building deliberately, because they are what the demo shows:

- **Barge-in.** When the VAD detects the user speaking while the agent is talking, cancel the in-flight LLM request *and* stop TTS playback immediately. Triton supports request cancellation; use it. Without cancellation the GPU keeps generating tokens nobody will hear, which is both a latency and a cost problem.
- **Endpointing is a product decision, not a default.** How long a silence means “your turn” determines whether the agent feels attentive or impatient. Make it configurable and show it in the Product tab.
- **Speculative first response.** Optionally begin generating on the ASR partial hypothesis and discard if the final transcript diverges. This buys real latency and costs wasted tokens — measure both before committing.

5. Frontend

Per §2 of the shared requirements. Project-specific detail:

Product tab — a call interface: push to talk or open mic, live waveform, live partial transcript, the agent’s reply as both audio and text, and a **turn-latency readout broken into stages** (VAD □ ASR □ LLM TTFT □ TTS first chunk). That breakdown is the single most persuasive thing on screen for a technical buyer, because it shows exactly where the budget goes.

Configuration: agent persona, company policies, escalation rules, endpointing silence threshold, voice selection.

Architecture tab — 3D graph. Logos: NVIDIA, TensorRT, Triton, Kubernetes, Prometheus, Grafana, PostgreSQL. The nodes needing the strongest cost/quality copy are **in-flight batching**, **paged KV cache**, **FP8 engine** and **barge-in cancellation**.

Monitoring tab — see below.

The Architecture tab must show the improvement loop. It is a stage card like any other, and it is the only one whose edges flow *back* upstream — that is the visual point. Give it the green treatment and its own tour stops; see §6 for the node inventory and 00-shared-requirements.md §2 for the graph mechanics (expandable pipeline, click-to-isolate, draggable nodes, canvas-drawn icon glyphs, constant-size labels, HTML detail panel).

The Monitoring tab has five sections, not four — Quality, Performance, Security, Audit and **Improvement**. The last one is never seeded with demo data.

6. Architecture graph: node inventory

These are the stage cards for this project’s Architecture tab, in pipeline order. Every one is a node; every child is a node inside its parent’s card. Build them against the ArchNode shape in `00-shared-requirements.md` §8a, with all four rationale fields and a “Say this” line on each.

#	Stage card	Colour role	Expands into
0	Caller	neutral	— (the person on the line)
1	Voice Client	teal	Mic capture, WebRTC transport, playback, barge-in detector
2	Edge & Security	amber	TLS + TURN, rate limiting, authentication (OIDC), authorization (RBAC), spoken-injection detection
3	Turn Detection	violet	VAD, endpointing threshold, streaming
4	Context & Skills	violet	ASR, partial hypotheses Intent analysis, RAG retrieval, account lookup, booking/transfer tools, prompt compiler, guardrails
5	TensorRT-LLM Engine	orange	Llama-3.1-8B-Instruct (FP8) , in-flight batching, paged KV cache (FP8), fused kernels & AOT engine build, request cancellation
6	Speech Output	teal	Streaming TTS, chunked synthesis, playback buffer
7	Database	blue	PostgreSQL, transcript store, model/engine registry
8	Monitoring & Audit	magenta	Prometheus, Grafana, hash-chained audit log, consent records, structured logging, alerting
9	Improvement Loop	green	Turn rating, A/B voice comparison, preference dataset, answer scoring, DPO fine-tune, promotion gate ☐
10	Platform	grey	<code>lm_eval</code> , GuideLLM Kubernetes, HPA on active-call count, staging ☐ production

The card that carries the commercial argument is **TensorRT-LLM Engine**. Its request cancellation child is the one most people have never thought about: barge-in means abandoning generation constantly, and without cancellation the GPU keeps producing tokens nobody will hear — a latency *and* a cost problem.

7. The improvement loop for this project

00-shared-requirements.md §5a applies in full. What is specific here:

The judgement is about a **spoken turn**, and the modality changes what is cheap to collect:

- **Rating** — a thumb on the agent's reply, in the transcript view. Callers will not press it; the operator reviewing recordings will.
- **Preference** — the same caller question, answered twice under different sampling settings, played back side by side. Both sides get the identical retrieved context and the identical transcript, so the judgement is about generation rather than about which side heard the question better.
- **Correction** — the reply the agent *should* have given. On a voice product this is the most valuable signal, because the failures are usually policy failures rather than fluency failures.

Voice-specific rule: **judge on the transcript, train on the transcript**. Do not put audio in the training set. It is special-category data under GDPR Art. 9 in several readings, it carries consent obligations the text does not, and the LLM never saw it anyway — it saw ASR output.

7a. LoRA adapters: where they earn their place here

LoRA is already in this project's improvement loop as the vehicle for DPO. It earns a second, larger role here: **one base engine, many personas**.

Test this: per-tenant voice personas on one GPU. A voice product is usually sold to several clients at once, each wanting a different persona, tone and policy set. The naive answer is a deployment per client, which multiplies the GPU bill by the customer count. The right answer is one base engine plus a small adapter per tenant, selected per request.

Approach	GPU cost for 20 tenants	Switching cost
One deployment per tenant	20×	n/a
Prompt-only differentiation	1×	free, but weak — tone drifts and policy adherence is inconsistent
Base engine + per-tenant LoRA	1×	adapter swap per request

The TensorRT-LLM caveat, and state it plainly. Engines here are compiled ahead of time, so LoRA support has to be built into the engine (`--lora_plugin`, with `max_lora_rank` and the adapter slots declared at build time). You cannot add an adapter to an engine that was not built for one. Decide the maximum rank and slot count before the build, and put that constraint in the architecture graph — it is exactly the kind of engineering detail a technical buyer uses to judge whether you have actually run this in production.

Measure before believing. Adapter application is not free at inference time. Benchmark p95 TTFT with adapters active against the base engine, at realistic concurrency and with adapters actually being swapped between requests. On a sub-800 ms turn budget, a 30 ms regression is 4% of the budget and it must be a deliberate choice rather than a surprise.

8. The release gate for this project

00-shared-requirements.md §5b applies in full: two axes, either one able to block the release on its own.

Dimension	Question	Tool	Thresholds that matter here
Quality	Is it still right?	<code>lm_eval</code> (instruction following) + held-out spoken-support suite scored on policy adherence and correct escalation	Instruction following is the tightest bound; it predicts whether the agent stays inside supplied policy

Dimension	Question	Tool	Thresholds that matter here
Performance	Is it still fast?	GuideLLM against Triton at a realistic concurrent-call rate	p95 TTFT is the gate that matters — it sits directly inside the 800 ms turn budget, so a 20% TTFT regression is a product failure, not a slow-down

Also re-measure **word error rate through the full ASR → LLM → TTS loop**, not on clean text. Compression damage shows up differently when the input is already noisy from ASR.

9. Monitoring and KPIs

The headline chart is a stacked turn-latency budget, p50/p95/p99, split by stage. Everything else is secondary.

Section	Metrics
Quality	Task completion rate, escalation rate and reasons, ASR WER, interruption rate (users talking over the agent — the honest proxy for “does it feel slow”)
Performance	Turn latency by stage, LLM TTFT and inter-token latency, tokens/sec, KV cache utilisation, in-flight batch occupancy, concurrent calls per GPU
Security	Prompt injection attempts (spoken injection is real — “ignore your instructions” said aloud), auth failures, rate-limit trips, 4xx/5xx
Audit	Call records with consent capture, retention state, redaction applied
Improvement (RLHF)	Turn approval rate, corrections captured, challenger win rate in A/B playback, preference pairs awaiting the next training run

The Improvement section is never seeded with demo data. Every other section falls back to synthetic numbers when the metrics backend is absent and says so with a badge. Feedback is a claim about what real people judged; a plausible approval rate nobody gave is the one number here that cannot be corrected by waiting for real traffic. An empty loop renders as empty.

Autoscale on queue depth or active-call count, never CPU. Alert when p95 turn latency crosses 800 ms, and when the interruption rate rises — the latter usually moves before users complain.

10. Security, audit and compliance

Voice adds obligations text does not have:

- **Recording consent must be captured and logged before audio is retained.** In many jurisdictions this is a legal precondition, not a nicety.
- **Voice is biometric data** under GDPR Art. 9 in several readings. Treat raw audio as special-category: minimise retention, encrypt with a separate key, and support erasure. Prefer discarding audio and keeping only the redacted transcript.
- **Spoken prompt injection** is a live attack surface. Detect and count it on the transcript, exactly as a text system would.
- Transcripts are PII-redacted on the write path.

11. Deployment

Kubernetes. GPU node pool with L40S or H100 — FP8 requires Hopper or Ada; on Ampere the engine falls back to INT4 and the latency target needs re-measuring.

The engine build is a **separate, versioned artifact** from the application image. It is large, slow to produce and specific to a GPU architecture; coupling it to application deploys would make every code change a multi-gigabyte push. Ship it as its own image, staged into the serving pod by an init container.

Warm minimum replicas of at least one. A cold TensorRT engine load is minutes, and a voice call cannot wait.

12. Deliverables

- Working browser voice interface with barge-in
- Engine build pipeline with FP8 quantization and a quality gate
- All three frontend tabs
- Grafana dashboards: turn-latency budget and product KPIs
- Helm charts and infrastructure as code
- Load test producing **measured** turn latency at concurrency
- README with the real numbers, not projections

Acceptance: 1. p95 turn latency under 800 ms at 10 concurrent calls, measured 2. Barge-in cancels generation within 200 ms of detected speech 3. Quality gate fails when a deliberately over-quantized engine is supplied 4. Architecture tab renders and the guided tour runs with the backend stopped

13. Out of scope

Telephony (SIP/PSTN) integration — browser WebRTC only. Speaker diarisation. Multilingual, beyond confirming the model does not break on accented English. Voice cloning, which carries consent problems disproportionate to a demo.

Project 02 — Document & Image Analysis (SGLang)

Read `00-shared-requirements.md` first. Everything in it applies here.

0. Numbered build plan

Paste this section into Claude Code's plan mode to scope the work. Each step is independently reviewable; steps 1–4 must land before the demo is filmable.

1. **Repository skeleton.** Monorepo with `frontend/`, `backend/`, `model/`, `deploy/`, `observability/`. Strict TypeScript, typed Python, lint + types + tests in CI from the first commit.
2. **SGLang serving layer.** Stand up the engine with the compression spec in §3–§4 below. Record weights and latency before and after, and state what the freed resource buys in the units the client pays for.
3. **Backend API.** The component chain in §5 as real services: authentication, authorization, adversarial-input detection, context assembly, inference, guardrails, structured logging, hash-chained audit log.
4. **Product tab.** The working demo, usable by someone who has never seen it, plus the configuration surface. Configuration changes must visibly change behaviour and be inspectable before they apply.
5. **Architecture tab.** The 3D expandable-pipeline graph, built to the spec in `00-shared-requirements.md` §2. Use the node inventory in §6 of this document as the content. Click-to-isolate, draggable nodes, canvas-drawn icon glyphs, constant-size labels, HTML detail panel with all four rationale fields plus a "Say this" line.
6. **Guided tour.** An ordered walk covering every node that carries the commercial argument, including the honest stop about what is *not* enabled. Arrow keys, autoplay, and it must run with the backend stopped.
7. **Monitoring tab.** Quality, Performance, Security, Audit, Improvement. Live data when available, seeded demo data otherwise, honest source badge — except Improvement, which is never seeded.
8. **Feedback capture.** Rating, side-by-side comparison and correction in the Product tab, per §7. Redact on write, stamp the model/config version, store the triple denormalised, mark what an export consumed.
9. **Export and training path.** JSON Lines in the trainer's own format, resumable. Training is operator-run, never automatic. Write a training manifest next to every artifact. Evaluate **LoRA adapters** per §7a — that section says where they belong in *this* project and where they do not.
10. **Release gate.** Both axes per §8 — quality and performance, each able to fail the build on its own. Wire it into CI.
11. **Deployment.** Containers, infrastructure as code, staging → production with a rollback, and autoscaling on the signal that actually saturates.
12. **README.** A stranger can reach a working demo. Ends with the cost/performance/accuracy tradeoff table, one row per real decision, and a portability section.

Acceptance is §12 of this document plus the shared deliverables checklist.

1. Brief

Build an **automated document understanding service**: upload an invoice, a claim form, an ID card or a shipping manifest, and get back **validated structured JSON** plus a visual overlay showing where on the page each field was read from.

Not a chat interface over images. A pipeline that a finance or operations team runs thousands of documents through daily, whose output feeds a downstream system, and whose errors cost money.

Two properties make it credible rather than a toy:

Every extracted field carries a bounding box and a confidence score, and the service refuses rather than guesses below a threshold.

A field with no provenance cannot be checked by the human reviewing it. A pipeline that always returns a value silently converts a low-confidence guess into a downstream error — which is exactly how document AI projects fail in production.

Target user: a company doing high-volume manual data entry from documents — accounts payable, insurance claims intake, logistics, KYC onboarding.

2. Why SGLang for this workload

SGLang's advantages are unusually well matched here, and both are about *repetition*:

- **RadixAttention.** SGLang keeps a radix tree of KV cache across requests, so any shared prefix is reused automatically — not just an exact-match prefix, but any common branch. Document extraction is the ideal case: every request for a given document type shares the same long system prompt and the same schema instructions. Processing ten thousand invoices means that prefix is prefilled once, not ten thousand times. On a long extraction prompt this is the dominant cost saving in the whole system, and it is the number to put in front of the client.
- **Structured output with constrained decoding.** SGLang can enforce a JSON schema or regex during generation, at the decoding level. The model *cannot* emit invalid JSON — it is not asked politely and then parsed hopefully. For a pipeline feeding a downstream system, this removes an entire class of production failure, and it is faster than generate-then-retry.
- **Its frontend language** expresses multi-step control flow — extract, then branch on document type, then extract type-specific fields — inside the runtime rather than across multiple round trips.

Compare honestly in the README: vLLM also has prefix caching, but SGLang's radix tree handles *branching* prefixes better, which is what a multi-document-type workload actually produces. Say where the advantage is real and where it is marginal.

3. Model and compression

Component	Choice
Vision-language model	Qwen2.5-VL-7B-Instruct (strong document OCR and grounding)
Alternative	InternVL2-8B
Quantization	W4A16 (GPTQ/AWQ) or FP8 on Hopper/Ada
Embeddings	bge-m3 for the document-type classifier and few-shot example retrieval

Compression spec - Quantize with a **document-domain calibration set** — real invoices, forms and scans, not generic web text. Quantization scales are fitted to the tokens the model sees; calibrating on the wrong distribution leaves accuracy on the table. Vision-language models are particularly sensitive here because the visual token distribution is nothing like web text. - Record weights before and after and state what the freed memory buys: longer document context and more pages in flight per GPU.

Quality gate — field-level, not document-level - Per-field precision and recall against a labelled holdout set, gated as a delta against the uncompressed baseline. - **Bounding box IoU** against ground truth — a correct value attributed to the wrong region is still a failure, because the human reviewer cannot verify it. - Schema conformance rate (should be 100% with constrained decoding — if it is not, the constraint is misconfigured). - **Calibration check:** confidence scores must be meaningfully ordered. A model that reports 0.9 on wrong answers is worse than one that reports 0.5 on everything, because the refusal threshold stops working.

4. Architecture

Upload (Product tab) → Edge (TLS, size limits, MIME validation, rate limit)
→ Authentication → Authorization (per document type)
→ Virus scan + format normalisation
→ Page splitting, skew, resolution normalisation
→ Document type classifier
→ RAG: schema + few-shot examples for that type
→ Skills: field validators (VAT format, IBAN checksum, date sanity, arithmetic totals)
→ SGLang runtime

- RadixAttention prefix reuse
- constrained JSON decoding
- quantized VLM

- Confidence gate → human review queue
- Stateless JSON logs
- Audit log (hash-chained, HIPAA/SOC2/GDPR)
- Prometheus → Grafana
- Staging / Production → Kubernetes + HPA

== RLHF / IMPROVEMENT LOOP ==

Field-level correction in the review UI (the correction IS the label) + accept/reject on whole extractions

- A/B: same document, two prompts, differing fields highlighted
- Preference dataset (region, extracted, corrected)
- Extraction scoring → Grafana
- DPO fine-tune (LoRA, operator-run, never automatic)
- Promotion gate: `lm_eval` (quality) + GuideLLM (performance)

==> back to the SGLang runtime

Points worth building deliberately:

- **Validators are code, not prompts.** An IBAN checksum, a VAT format, whether line items sum to the stated total — these are deterministic and must be computed, not asked of a model. When a validator disagrees with the model, that is the highest value signal in the system: route it to human review.
- **The human review queue is part of the product, not an admission of failure.** Show it. A client evaluating this is comparing against 100% manual entry; a system that handles 85% automatically and routes 15% with the uncertain fields highlighted is an enormous win, and being honest about the 15% is more persuasive than claiming 100%.
- **Batch endpoint** for the real workload. Interactive upload is the demo; the business case is ten thousand documents overnight, where RadixAttention's prefix reuse compounds.

5. Frontend

Product tab — split view: document on the left with **bounding box overlays**, extracted JSON on the right. Hovering a JSON field highlights its box on the page and vice versa. Low-confidence fields are visually flagged and editable, and a correction is recorded as training signal.

That linked hover is the single most convincing interaction in this project. It turns “the model said 1,240.50” into “the model read 1,240.50 from *here*”, which is the difference between a claim and evidence.

Configuration: document types, per-type JSON schema, confidence thresholds, validation rules, review routing.

Architecture tab — 3D graph. Logos: SGLang, NVIDIA, Kubernetes, Prometheus, Grafana, PostgreSQL, Qwen. The nodes needing the strongest copy are **RadixAttention, constrained decoding, the confidence gate and the validators**.

Monitoring tab — see below.

The Architecture tab must show the improvement loop. It is a stage card like any other, and it is the only one whose edges flow *back* upstream — that is the visual point. Give it the green treatment and its own tour stops; see §6 for the node inventory and `00-shared-requirements.md` §2 for the graph mechanics (expandable pipeline, click-to-isolate, draggable nodes, canvas-drawn icon glyphs, constant-size labels, HTML detail panel).

The Monitoring tab has five sections, not four — Quality, Performance, Security, Audit and **Improvement**. The last one is never seeded with demo data.

6. Architecture graph: node inventory

These are the stage cards for this project's Architecture tab, in pipeline order. Every one is a node; every child is a node inside its parent's card. Build them against the ArchNode shape in `00-shared-requirements.md` §8a, with all four rationale fields and a “Say this” line on each.

#	Stage card	Colour role	Expands into
0	User	neutral	— (uploads a document)
1	Upload Console	teal	Drag-and-drop, page preview, extraction review, schema editor
2	Edge & Security	amber	TLS, rate limiting, authentication, authorization, file-type and size validation, malicious-document scanning
3	Document Pipeline	violet	Page splitting, OCR, layout detection, region ordering, image normalisation
4	Context & Schema	violet	Target schema, few-shot exemplars, retrieval of prior extractions, prompt compiler, injection defence on document text
5	SGLang Runtime	orange	Qwen2.5-VL-7B-Instruct (INT4/FP8) , RadixAttention prefix reuse, structured/constrained decoding, continuous batching, paged KV cache
6	Validation	violet	Schema conformance, confidence gating, human-review routing
7	Database	blue	PostgreSQL + pgvector, extraction store, document registry
8	Monitoring & Audit	magenta	Prometheus, Grafana, hash-chained audit log, structured logging, alerting
9	Improvement Loop	green	Field-level correction capture, A/B extraction comparison, preference dataset, extraction scoring, DPO fine-tune, promotion gate □
10	Platform	grey	lm_eval, GuideLLM, Kubernetes, HPA on queue depth, staging □ production

The card carrying the argument is **SGLang Runtime**, and specifically **RadixAttention**: every document in a batch shares the same schema prompt and the same few-shot exemplars, so the shared prefix is enormous and prefix reuse is worth far more here than in a chat workload. Make the detail panel say what fraction of the prompt is shared.

7. The improvement loop for this project

00-shared-requirements.md §5a applies in full. What is specific here:

Document extraction has the best feedback signal of any project in this pack, because **the correction is the label**:

- **Field-level correction** — the reviewer fixes one extracted value in the review UI. That edit is simultaneously a bug report, a training example and an accuracy measurement. Capture it as (document region, extracted, corrected).
- **Preference** — the same document extracted twice under different prompts or sampling settings, shown side by side with the differing fields highlighted.
- **Rating** — accept / reject on a whole extraction, for volume.

Rule specific to this domain: **store the page region, not just the text**. A correction without the region it came from cannot be used to train a vision-language model, and reconstructing it later is unreliable once the document pipeline changes.

7a. LoRA adapters: where they earn their place here

This is the project where LoRA pays best, so build it and demonstrate it.

Document extraction is naturally multi-tenant and naturally narrow: every client has their own document types, their own schema and their own conventions about what “invoice date” means. That is precisely the shape a small adapter fits, and precisely the shape a single general model handles poorly.

Test this: one adapter per document type, served from one base model.

Approach	GPU cost for 15 document types	Accuracy on a narrow type
One general model, prompt-only	1×	worst — the model hedges across conventions it has seen
Fine-tuned model per type	15×	best, and unaffordable
Base model + LoRA per type	1×	close to per-type fine-tuning

SGLang serves multiple adapters against one base model, so the routing decision is “which adapter for this document type”, made at request time. Build that routing as a visible node in the architecture graph — “one GPU, fifteen specialisms” is the clearest cost story in this entire pack.

Where the adapters come from. The improvement loop already produces them: field-level corrections on invoices train the invoice adapter, not a global model. That containment is a feature — a bad batch of corrections on one document type cannot degrade extraction on the other fourteen. Say that explicitly; it is the answer to “what if the feedback data is bad”.

Measure: per-adapter extraction accuracy against the shared base, adapter swap latency, and prefix-cache hit rate with adapters active — check that adapter switching has not quietly destroyed prefix sharing, which would leave accuracy untouched and multiply the bill.

8. The release gate for this project

00-shared-requirements.md §5b applies in full: two axes, either one able to block the release on its own.

Dimension	Question	Tool	Thresholds that matter here
Quality	Is it still right?	lm_eval + held-out extraction suite scored field-by-field against ground truth	Per-field exact-match and schema-conformance rate. An aggregate accuracy hides the case where one critical field degraded and forty trivial ones improved
Performance	Is it still fast?	GuideLLM at a realistic document arrival rate, with realistic prompt lengths	Prefix-cache hit rate is the cost metric. A change that breaks prefix sharing can leave quality untouched and multiply the bill

Benchmark with **real page counts and real prompt lengths**. A benchmark run with a 128-token prompt measures a workload this system never serves.

9. Monitoring and KPIs

Section	Metrics
Quality	Per-field precision/recall, bounding box IoU, schema conformance, straight-through processing rate, human correction rate by field
Performance	Latency per page p50/p95/p99, pages/sec, RadixAttention cache hit rate , KV cache utilisation, batch occupancy, GPU utilisation
Security	Malicious upload attempts, prompt injection embedded in document text (a real and under-appreciated attack — text inside an uploaded PDF instructing the model), auth failures, 4xx/5xx
Audit	Every extraction: who, which document, which model version, which config version, what was returned, who corrected it
Improvement (RLHF)	Field-level correction rate (per field), extraction approval rate, challenger win rate, preference pairs awaiting the next training run

The Improvement section is never seeded with demo data. Every other section falls back to synthetic numbers when the metrics backend is absent and says so with a badge. Feedback is a claim about what real people judged; a plausible approval rate nobody gave is the one number here that cannot be corrected by waiting for real traffic. An empty loop renders as empty.

The cost metric that sells the project: cost per document processed, next to the loaded cost of manual entry. Include the human review cost honestly — the number is still overwhelming, and an honest number survives scrutiny.

Watch the RadixAttention hit rate. A drop means prefixes stopped being shared — usually because someone put a per-request value (a timestamp, a document ID) at the front of the prompt. That single mistake can multiply serving cost several-fold and is invisible without this metric.

10. Security, audit and compliance

- **Documents are the attack surface.** Text inside an uploaded PDF instructing the model to ignore its schema is a working attack. The system prompt must state that document content is data, never instructions, and the pipeline must detect and count injection attempts found in extracted text.
- Uploaded documents routinely contain special-category data — health claims, identity documents. Encrypt with a dedicated key, define retention, support erasure, and keep source documents out of any general-purpose data store.
- Audit every extraction with model version and config version. When a downstream system acts on a wrong value six months later, this is the only way to reconstruct what happened.
- Per-document-type authorization: a user cleared for invoices should not be able to submit medical claims.

11. Deployment

Kubernetes with a GPU node pool. VLMs need more memory than a text model of the same parameter count — image tokens are numerous — so size for that and verify before committing to an instance type.

Two serving paths worth separating: an interactive endpoint tuned for latency, and a batch endpoint tuned for throughput with a much larger batch size. They have opposite optimal settings, and forcing one configuration to serve both leaves a large amount of throughput unused.

12. Deliverables

- Upload  structured JSON with bounding boxes and confidence
- Batch processing endpoint
- Human review queue with correction capture
- Quantization pipeline with a field-level quality gate
- All three frontend tabs
- Grafana dashboards including RadixAttention hit rate
- Helm charts and infrastructure as code
- Measured throughput and cost per document

Acceptance: 1. Field-level precision above the agreed threshold on a labelled holdout set 2. 100% schema conformance via constrained decoding 3. Injection text embedded in an uploaded document is detected, counted, and does not alter extraction behaviour 4. RadixAttention hit rate demonstrably above 60% on a homogeneous batch 5. Architecture tab and guided tour run with the backend stopped

13. Out of scope

Handwriting recognition beyond what the base model does. Training a custom detector. Non-Latin scripts, beyond confirming nothing crashes. Direct ERP integration — expose an API and stop there.

Project 03 — Offline Field Assistant (llama.cpp)

Read `00-shared-requirements.md` first. Everything in it applies here, with the device-class substitutions in §4.

0. Numbered build plan

Paste this section into Claude Code's plan mode to scope the work. Each step is independently reviewable; steps 1–4 must land before the demo is filmable.

1. **Repository skeleton.** Monorepo with `frontend/`, `backend/`, `model/`, `deploy/`, `observability/`. Strict TypeScript, typed Python, lint + types + tests in CI from the first commit.
2. **llama.cpp serving layer.** Stand up the engine with the compression spec in §3–§4 below. Record weights and latency before and after, and state what the freed resource buys in the units the client pays for.
3. **Backend API.** The component chain in §5 as real services: authentication, authorization, adversarial-input detection, context assembly, inference, guardrails, structured logging, hash-chained audit log.
4. **Product tab.** The working demo, usable by someone who has never seen it, plus the configuration surface. Configuration changes must visibly change behaviour and be inspectable before they apply.
5. **Architecture tab.** The 3D expandable-pipeline graph, built to the spec in `00-shared-requirements.md` §2. Use the node inventory in §6 of this document as the content. Click-to-isolate, draggable nodes, canvas-drawn icon glyphs, constant-size labels, HTML detail panel with all four rationale fields plus a “Say this” line.
6. **Guided tour.** An ordered walk covering every node that carries the commercial argument, including the honest stop about what is *not* enabled. Arrow keys, autoplay, and it must run with the backend stopped.
7. **Monitoring tab.** Quality, Performance, Security, Audit, Improvement. Live data when available, seeded demo data otherwise, honest source badge — except Improvement, which is never seeded.
8. **Feedback capture.** Rating, side-by-side comparison and correction in the Product tab, per §7. Redact on write, stamp the model/config version, store the triple denormalised, mark what an export consumed.
9. **Export and training path.** JSON Lines in the trainer's own format, resumable. Training is operator-run, never automatic. Write a training manifest next to every artifact. Evaluate **LoRA adapters** per §7a — that section says where they belong in *this* project and where they do not.
10. **Release gate.** Both axes per §8 — quality and performance, each able to fail the build on its own. Wire it into CI.
11. **Deployment.** Containers, infrastructure as code, staging → production with a rollback, and autoscaling on the signal that actually saturates.
12. **README.** A stranger can reach a working demo. Ends with the cost/performance/accuracy tradeoff table, one row per real decision, and a portability section.

Acceptance is §12 of this document plus the shared deliverables checklist.

1. Brief

Build an **on-device assistant for field workers who have no reliable network**: technicians in plant rooms and basements, maintenance crews on remote sites, clinicians in facilities that forbid cloud upload, inspectors in the field.

It answers questions from a large body of technical documentation — service manuals, safety procedures, part specifications — with everything running locally: the model, the retrieval index, and the audit log.

The defining property:

The device runs in airplane mode for the entire demo. No request leaves it.

That is the product. Not “privacy-conscious” or “encrypted in transit” — the data physically cannot leave, because there is nowhere for it to go. For a client in a regulated industry or a security-cleared site, that is a different category of argument from any cloud assurance.

Target user: industrial services, utilities, defence, healthcare facilities management — organisations whose people work where connectivity is absent and whose data cannot go to a third party regardless.

2. Why llama.cpp for this workload

- **It runs where nothing else does.** llama.cpp is C++ with minimal dependencies and mature backends for Metal (iOS/macOS), Vulkan and NNAPI (Android), and CPU everywhere. No Python runtime, no CUDA, no container.
- **GGUF quantization is the enabling technology, not an optimisation.** A 7B model at Q4_K_M is roughly 4.4 GB against ~14 GB at FP16. On a phone this is not a cost saving — it is the entire difference between the product existing and not existing. That framing belongs in the architecture graph.
- **mmap model loading** means the OS pages weights in on demand. Startup is fast and the app does not need to hold the whole model resident, which matters enormously under mobile memory pressure where the OS kills large processes.
- **It does have a KV cache**, and on-device it is a visible constraint rather than a background detail: context length is bounded by RAM you can actually feel. This makes it an unusually good thing to *demonstrate* — show context filling up and the memory cost rising in real time.

The honest limitation, state it plainly: a 7B model quantized to 4 bits is meaningfully weaker than a frontier model. The system must be designed around that — strong retrieval, narrow scope, and a low threshold for saying “I don’t know”. A project that pretends otherwise will fail its first real user.

3. Model and compression

Component	Choice
LLM	Qwen2.5-3B-Instruct or Llama-3.2-3B-Instruct
Format	GGUF, Q4_K_M primary; ship Q5_K_M and Q8_0 for comparison
Embeddings	bge-small-en-v1.5 GGUF, also on-device
Vector store	SQLite with sqlite-vec, or a flat index — the corpus is bounded

Start at 3B, not 7B. On a mid-range phone a 7B model at Q4 leaves too little headroom for the OS, and the app gets killed in the background. Ship 3B as default and offer 7B as an opt-in for high-end devices. Measure both.

Compression spec - Produce Q4_K_M, Q5_K_M and Q8_0 variants. Record for each: file size, RAM at runtime, tokens/sec on target hardware, and quality-gate score. - **Put that table in the README and in the architecture graph node.** The size/speed/quality trade is the most concrete engineering decision in the project, and showing the measured curve is far more persuasive than asserting a choice. - Consider an importance matrix (imatrix) computed on domain documentation to improve low-bit quality — same principle as calibration data elsewhere in this pack.

Quality gate - Domain suite of held-out questions with reference answers from the documentation, scored on: does it answer from the retrieved passage, does it cite, does it decline when the answer is absent. - Run the gate **on the target device**, not on a workstation. Quantized kernels differ per backend, and Metal, Vulkan and CPU do not produce identical output. A gate that only ran on a laptop has not tested what ships. - Track tokens/sec and time-to-first-token per device tier as gated metrics — correctness that arrives too slowly is still a failure on mobile.

4. Architecture

Same chain as 00-shared-requirements.md §3, collapsed onto one device. Every component still exists; several change form, and the graph should say so explicitly.

```
User (Product tab, native or React Native shell)
→ Device authentication (biometric / passcode)    ← replaces 0IDC
→ Local authorization (which document sets)       ← role baked into provisioning
→ Query analysis
→ On-device retrieval (SQLite + vector index)
→ Skills: unit conversion, part lookup, checklists – all local
→ llama.cpp
```

- GGUF Q4_K_M quantized weights
- KV cache (RAM-bounded – surface it)
- Metal / NNAPI / Vulkan offload
- Guardrails + confidence gate
- Local structured logs (stateless between sessions)
- Local append-only audit log, hash-chained
- Local metrics buffer
 - … opportunistic sync when connectivity returns …
- [Backend, when reachable] Prometheus → Grafana
- Model registry → staged OTA rollout rings ← replaces Kubernetes
 - Release rings (signed GGUF, staged OTA)

== RLHF / IMPROVEMENT LOOP ==

- On-device rating + correction, written to a LOCAL queue
- redacted on device, before the queue
 - upload is opt-in, explicit, per item – the queue is visible to the user, who can delete any of it
 - Preference dataset (prompt, chosen, rejected)
 - DPO fine-tune (LoRA, operator-run, never automatic)
 - Promotion gate: lm_eval + on-device latency/memory harness

==> back to the shipped GGUF

Substitutions to state explicitly on the nodes (see shared requirements §4):

- **Kubernetes** □ **model registry with staged OTA rollout.** A phone does not run Kubernetes. The requirement is a controlled path from staging to production with rollback: signed model artifacts, versioned, released to a canary ring, then 10%, then general – with the ability to halt and roll back a bad model.
- **Server monitoring** □ **local buffer with opportunistic sync.** Metrics and audit events are written locally and uploaded when a network appears. They must survive app restarts and be capped so a device offline for weeks does not fill its storage.
- **Continuous batching** □ **not applicable.** Single user, one request at a time. Say so on the node rather than omitting it. The on-device analogue worth demonstrating is mmap loading and KV cache growth.

5. Frontend

React Native (or native shell with a React web view) so the three-tab structure is consistent with the rest of the pack.

Product tab — the assistant. Answers with citations into the source manual, page and section. Offline indicator prominently displayed. A **live resource readout**: tokens/sec, RAM in use, KV cache occupancy, battery drain.

That readout is not developer garnish — on mobile it is the product's credibility. A client's first question is “will this destroy the battery and fill the phone?”, and showing the answer live is the strongest possible response.

Configuration: document sets, model variant (Q4/Q5/Q8) with the trade-off table visible, context length, confidence threshold.

Architecture tab — 3D graph. Logos: llama.cpp, GGUF, SQLite, Apple Metal, Android, Prometheus, Grafana. Strongest copy needed on **GGUF quantization, mmap loading, KV cache, on-device retrieval and OTA rollout rings**.

Note: a 3D force graph on a phone needs a reduced node count and a lower frame rate cap, or it will drain battery visibly during the demo. Consider rendering the Architecture tab full-fidelity on tablet and a simplified layout on phone.

Monitoring tab — see below.

The Architecture tab must show the improvement loop. It is a stage card like any other, and it is the only one whose edges flow *back* upstream — that is the visual point. Give it the green treatment and its own tour stops; see §6 for the node inventory and `00-shared-requirements.md` §2 for the graph mechanics (expandable pipeline, click-to-isolate, draggable nodes, canvas-drawn icon glyphs, constant-size labels, HTML detail panel).

The Monitoring tab has five sections, not four — Quality, Performance, Security, Audit and **Improvement**. The last one is never seeded with demo data.

6. Architecture graph: node inventory

These are the stage cards for this project’s Architecture tab, in pipeline order. Every one is a node; every child is a node inside its parent’s card. Build them against the `ArchNode` shape in `00-shared-requirements.md` §8a, with all four rationale fields and a “Say this” line on each.

#	Stage card	Colour role	Expands into
0	User	neutral	— (asks, offline, on a plane)
1	Mobile App	teal	Chat UI, streaming renderer, model download manager, storage budget
2	Local Trust Boundary	amber	On-device authentication (biometric unlock), per-app sandbox, no network path — say this loudly, it is the product
3	On-device Context	violet	Local document index, embedding lookup, conversation history, prompt assembly
4	llama.cpp Runtime	orange	Qwen2.5-3B-Instruct (GGUF Q4_K_M) , mmap weight loading, Metal / NNAPI offload, KV cache, thread and batch tuning
5	Output Guardrails	violet	Confidence gating, refusal on unknown, citation into local documents
6	Local Store	blue	SQLite, conversation history, encrypted at rest with the platform keystore
7	Telemetry & Audit	magenta	Opt-in , aggregated, on-device audit log, crash reporting, no content leaves the device
8	Improvement Loop	green	On-device rating and correction, local-only queue , opt-in upload, preference dataset, DPO fine-tune, promotion gate ☐ <code>lm_eval</code> , on-device latency harness

#	Stage card	Colour role	Expands into
9	Release Rings	grey	Model registry, signed GGUF artifacts, staged OTA rollout rings, rollback

Two cards carry this project’s argument and neither is the model. **Local Trust Boundary** is the reason a buyer chooses this over an API at all — nothing leaves the device — and **Release Rings** is the honest answer to “how do you ship an update to something with no server”. A phone does not run Kubernetes; the requirement is a controlled path with a rollback, not the specific tool.

7. The improvement loop for this project

00-shared-requirements.md §5a applies in full. What is specific here:

This is the hardest feedback loop in the pack, and being honest about why is worth more than pretending it is easy.

The device holds the signal and the network is the problem. A user who is offline by design cannot stream judgements to a server, and a product whose selling point is “nothing leaves the device” cannot quietly upload the conversations people corrected.

Design it as:

- **Collect locally, always.** Rating and correction are written to an on-device queue. This works offline and costs nothing.
- **Upload is opt-in, explicit, and per-item.** Not a settings toggle buried in a menu — a visible “send this correction to improve the assistant” action on the item itself. A blanket consent for a privacy-first product is a contradiction.
- **Redact on device, before the queue.** Not on the server. The point is that the raw text never leaves.
- **Show the queue.** Let the user see exactly what would be sent, and delete any of it. This is also the most persuasive privacy demo you can film.

Expect a low upload rate and say so in the README. A small volume of genuinely consented corrections is worth more than a large volume of quietly harvested ones, and it is the only version of this that survives a privacy review.

7a. LoRA adapters: where they earn their place here

Test this: the adapter is the update.

The hard problem in shipping an on-device model is not inference, it is distribution. A 4 GB GGUF re-download over mobile data is a thing users refuse, and it is the reason on-device assistants ship once and never improve.

A LoRA adapter is tens of megabytes. That changes what an update *is*:

Update type	Download	Realistic adoption
Full quantized model	~2–4 GB	poor — Wi-Fi only, deferred, often never
LoRA adapter	~20–80 MB	routine, like any app update

llama.cpp applies GGUF LoRA adapters at load time (or you can merge one into the base for a single artifact). Both paths are worth building and comparing:

- **Applied at load** — the base model stays on disk once, and each update ships only the adapter. Best distribution story, small runtime cost.
- **Merged ahead of time** — one artifact, no runtime overhead, but every update is a full re-download. Best inference story, worst distribution story.

Measure both and put the numbers in the README: load time, tokens/sec and peak RSS with an adapter

applied versus merged versus base. On the oldest device in the support matrix. If applying at load costs meaningful memory, that decides it — memory is the hard ceiling on a phone, and exceeding it does not make the app slow, it makes the OS kill it.

Also test: per-vertical adapters. A field assistant for electricians and one for plumbers can be the same 3B base with different adapters. That is a product strategy that costs one download plus a small file per vertical, and it is impossible with a monolithic model.

8. The release gate for this project

00-shared-requirements.md §5b applies in full: two axes, either one able to block the release on its own.

Dimension	Question	Tool	Thresholds that matter here
Quality	Is it still right?	lm_eval on the desktop build of the same GGUF + held-out on-device suite	Run the gate against the exact quantized GGUF that ships , not the fp16 source. Q4_K_M is where the damage happens
Performance	Is it still fast enough on the worst phone you support?	On-device latency harness: p95 time-to-first-token, tokens/sec, peak RSS , thermal throttling onset, energy per reply	Peak memory is the hard ceiling. Exceeding it does not make the app slow, it makes the OS kill it — an absolute bound, not a relative one

GuideLLM targets a server; substitute an on-device harness and say so. Measure on the **oldest device in the support matrix**, sustained for several minutes — a benchmark that stops before the phone gets hot measures a state the user never experiences.

9. Monitoring and KPIs

Section	Metrics
Quality	Answer accuracy against the domain suite, citation rate, decline rate, user thumbs-down rate
Performance	Tokens/sec, TTFT, model load time, peak RAM , KV cache size vs context, battery drain per query , thermal throttling events
Security	Failed device authentications, attempts to access unauthorised document sets, integrity check failures on model artifacts
Audit	Local hash-chained log: every query, which model version, which document set, sync status
Improvement (RLHF)	Local judgements captured, opt-in upload rate (expect it to be low — say so), challenger win rate, pairs awaiting the next training run

The Improvement section is never seeded with demo data. Every other section falls back to synthetic numbers when the metrics backend is absent and says so with a badge. Feedback is a claim about what real people judged; a plausible approval rate nobody gave is the one number here that cannot be corrected by waiting for real traffic. An empty loop renders as empty.

Battery and thermal are first-class metrics here, not curiosities. A model that is fast for ten minutes and then thermally throttles to unusability has failed, and this only shows up under sustained load on real

hardware. Measure it across a realistic session, not a single query.

Report per device tier. A flagship phone and a three-year-old mid-range device are different products, and the client needs both numbers.

10. Security, audit and compliance

The offline architecture is the security story — but it must be built, not claimed:

- **Model artifacts are signed and verified before load.** An unsigned model file is arbitrary code execution risk on a device you do not control.
- **Documents encrypted at rest** with a key in the platform keystore (Keychain / Android Keystore), released by biometric authentication.
- **The audit log is local, append-only and hash-chained**, and it syncs when connectivity allows. Tamper-evidence matters more here, not less, because the device is physically in an untrusted environment.
- **Remote wipe** for lost devices — a real requirement for regulated deployments and one clients ask about immediately.
- **GDPR:** on-device processing is a strong data-minimisation and lawful-basis story. State it accurately: data does not leave the device, so there is no transfer and no third-party processor for the inference step.

11. Deployment

- iOS via TestFlight, Android via Play internal → closed → open testing tracks. That progression *is* staging → production; map it explicitly in the graph.
- **Models ship separately from the app binary.** A 4.4 GB model inside an app bundle is a hostile download and makes every code fix a full re-download. Deliver models as signed, versioned artifacts fetched on first run and updated independently.
- Staged rollout rings with halt-and-rollback. A bad quantization reaching every device at once, on hardware you cannot reach, is the worst failure mode this project has.

12. Deliverables

- Working mobile app, fully functional in airplane mode
- GGUF quantization pipeline producing three variants with a measured comparison table
- Quality gate that runs **on-device**
- All three tabs
- Local audit log with hash chain and sync
- Model registry with signed artifacts and staged rollout
- Measured performance across at least two device tiers

Acceptance: 1. Full demo completes in airplane mode with no network calls (prove it with a packet capture, not an assertion) 2. Sustained session on a mid-range device without thermal collapse, with the curve recorded 3. Audit log hash chain verifies, and a tampered entry is detected 4. A model artifact with a bad signature is refused 5. Architecture tab and guided tour run offline

13. Out of scope

Multi-user on one device. Real-time sync between devices. Fine-tuning on-device. Voice input — that is Project 01. Corpora larger than device storage allows, which would need a fundamentally different retrieval design.

Project 04 — On-Device Face Verification (ExecuTorch)

Read `00-shared-requirements.md` first. Everything in it applies here, with the device-class substitutions in §4.

0. Numbered build plan

Paste this section into Claude Code's plan mode to scope the work. Each step is independently reviewable; steps 1–4 must land before the demo is filmable.

1. **Repository skeleton.** Monorepo with `frontend/`, `backend/`, `model/`, `deploy/`, `observability/`. Strict TypeScript, typed Python, lint + types + tests in CI from the first commit.
2. **ExecuTorch serving layer.** Stand up the engine with the compression spec in §3–§4 below. Record weights and latency before and after, and state what the freed resource buys in the units the client pays for.
3. **Backend API.** The component chain in §5 as real services: authentication, authorization, adversarial-input detection, context assembly, inference, guardrails, structured logging, hash-chained audit log.
4. **Product tab.** The working demo, usable by someone who has never seen it, plus the configuration surface. Configuration changes must visibly change behaviour and be inspectable before they apply.
5. **Architecture tab.** The 3D expandable-pipeline graph, built to the spec in `00-shared-requirements.md` §2. Use the node inventory in §7 of this document as the content. Click-to-isolate, draggable nodes, canvas-drawn icon glyphs, constant-size labels, HTML detail panel with all four rationale fields plus a “Say this” line.
6. **Guided tour.** An ordered walk covering every node that carries the commercial argument, including the honest stop about what is *not* enabled. Arrow keys, autoplay, and it must run with the backend stopped.
7. **Monitoring tab.** Quality, Performance, Security, Audit, Improvement. Live data when available, seeded demo data otherwise, honest source badge — except Improvement, which is never seeded.
8. **Feedback capture.** Rating, side-by-side comparison and correction in the Product tab, per §8. Redact on write, stamp the model/config version, store the triple denormalised, mark what an export consumed.
9. **Export and training path.** JSON Lines in the trainer's own format, resumable. Training is operator-run, never automatic. Write a training manifest next to every artifact. Evaluate **LoRA adapters** per §8a — that section says where they belong in *this* project and where they do not.
10. **Release gate.** Both axes per §9 — quality and performance, each able to fail the build on its own. Wire it into CI.
11. **Deployment.** Containers, infrastructure as code, staging → production with a rollback, and autoscaling on the signal that actually saturates.
12. **README.** A stranger can reach a working demo. Ends with the cost/performance/accuracy tradeoff table, one row per real decision, and a portability section.

Acceptance is §13 of this document plus the shared deliverables checklist.

1. Scope, stated first

This project builds **1:1 face verification** — confirming that the person in front of the camera is the enrolled owner of *this* device or *this* credential. It is the same category of function as a fingerprint unlock or a photo-matched building pass.

It is **not** a 1:N identification system, not a surveillance tool, and not a watchlist matcher. Three constraints enforce that in the design, and they are architectural rather than policy:

1. **Enrolment is explicit and consented**, initiated by the subject on the device.
2. **Templates never leave the device.** There is no central gallery to search against, so 1:N identification of strangers is not merely disallowed — it is not possible with what gets built.
3. **The enrolled set is one person per credential.** The matching API takes a probe and a single stored template. There is no “search all enrolled” call.

Build it this way. If a requirement arrives later to identify unknown people against a central database, that is a materially different system with different legal obligations, and it should be scoped, reviewed and consented separately.

Target user: workforce access control, secure app unlock, device-bound identity for regulated workflows — cases where the alternative is a shared PIN or a badge that can be handed over.

2. Why ExecuTorch for this workload

- **It is PyTorch's on-device runtime**, so the path from a trained model to a deployed one is short: export to .pte from the same PyTorch graph, without a lossy conversion through an intermediate format. For a vision pipeline that may need retraining on customer data, that shortens the loop considerably.
- **Delegate-based backends.** XNNPACK for CPU, Core ML and Metal on Apple, Vulkan and Qualcomm QNN on Android — chosen per device at export time, with the same source graph. A face embedding runs on the NPU where one exists and falls back gracefully where it does not.
- **Small runtime footprint.** ExecuTorch is designed for a constrained binary, which matters when the model must ship inside an app.
- **Ahead-of-time compilation and memory planning.** Buffer sizes are resolved at export, so runtime allocation is predictable — which is what makes per-frame latency stable rather than merely fast on average.

3. There is no KV cache here — and that is the point

Per shared requirements §4, state this explicitly on the architecture node rather than quietly omitting it:

A face embedding model runs **one forward pass per image**. There is no autoregressive generation, therefore no KV state, therefore no KV cache and no continuous batching. Fabricating one would be dishonest, and a client with domain knowledge will notice.

The optimisations that *do* apply, and which must be demonstrated in their place:

Technique	What it buys
INT8 post-training quantization	~4× smaller, faster on NPU/DSP which are integer-native
Quantization-aware training	Recovers accuracy INT8 PTQ loses — worth it, because face embeddings are sensitive to precision
Operator fusion	Conv+BN+ReLU folded — fewer memory round trips, which is the actual bottleneck
Delegate selection	NPU vs GPU vs CPU per device tier
Input resolution tuning	The single largest latency lever in vision, and a direct accuracy trade

That table is the honest analogue of the vLLM project's KV-cache story, and it belongs in the architecture graph in the same position.

4. Models and compression

Stage	Model	Notes
Face detection	BlazeFace or RetinaFace-mobile	Fast, single-face is enough
Landmark + alignment	5-point	Alignment before embedding matters more than model size
Embedding	MobileFaceNet or ArcFace (MobileNetV3 backbone)	128- or 512-d, L2-normalised
Liveness	Small anti-spoofing CNN	Mandatory — see §7

Compression spec - Export each stage to .pte with INT8 quantization. Calibrate on a **diverse** face dataset — see the fairness note below, because this is where bias enters. - Record per stage and per device tier: model size, latency, accuracy delta. - Choose delegates per target and record the fallback chain.

Quality gate — and this one carries an obligation the others do not - Verification accuracy: TAR at a fixed FAR (e.g. TAR@FAR=1e-4), gated as a delta against the FP32 baseline. - **Fairness evaluation across**

demographic groups, as a gating criterion, not a report. Face recognition accuracy is well documented to vary by skin tone, age and gender. A model that passes on aggregate while failing badly for one group is not shippable, and aggregate-only evaluation is how that ships anyway. Gate on the *worst-group* metric, not the mean. - Liveness: spoof detection rate against printed photos and screen replays. - On-device latency budget per device tier.

Run the gate on target hardware. INT8 kernels differ across delegates.

5. Architecture

User (Product tab)

- Camera capture
- Device authentication (passcode fallback)
- Authorization (which credential is being verified)
- Face detection → landmark alignment
- LIVENESS CHECK ← before embedding, always
- ExecuTorch embedding model
 - INT8 quantized .pte
 - NPU / GPU / CPU delegate
 - operator fusion
- 1:1 match against the single stored template (cosine, on-device)
- Confidence gate → accept / reject / fall back to passcode
- Local structured logs (no images, no templates)
- Local append-only audit log, hash-chained
- Metrics buffer ... opportunistic sync ...
- [Backend] Prometheus → Grafana
- Model registry → staged OTA rollout rings ← replaces Kubernetes
 - Release rings (signed artifacts, canary devices)

== IMPROVEMENT LOOP (the RLHF analogue) =====

False-reject and false-match reports from operator review

- embeddings and similarity SCORES only – never the frames
- Threshold review: projected TAR@FAR from stored score distributions. This is the loop that improves the product day to day, and it needs no training run at all
- Re-enrolment queue for poor-quality enrolments
- Promotion gate: TAR@FAR suite (per demographic group)
 - + on-device latency / energy harness

==> back to the embedding model

There is no preference fine-tune here by default, and that is deliberate: consent to be recognised is not consent to become training data.

Build deliberately:

- **The template is stored in the platform secure enclave** (Secure Enclave / StrongBox), never in app storage, never in a backup, never transmitted. This is the architectural claim the whole project rests on, and it must be true.
- **Liveness runs before embedding, unconditionally.** A verification system without anti-spoofing is defeated by a photograph, which makes it worse than a PIN because it carries an unearned appearance of security.
- **Passcode fallback always exists.** Biometrics fail — injuries, lighting, a changed appearance. A system with no fallback locks legitimate users out, and the failure correlates with exactly the groups the model already serves worst.
- **Images are never persisted.** Capture, embed, compare, discard. The audit log records that a verification occurred and its outcome, never the face.

6. Frontend

Product tab — enrolment flow with explicit consent, then verification. Show live: detection box, liveness verdict, match score against threshold, and the decision. Include a deliberate **spoofing demo** — hold a photo to the camera and watch liveness reject it. That is the most persuasive twenty seconds in the video.

Configuration: match threshold with the FAR/FRR trade shown as a curve rather than a number, liveness strictness, fallback policy, enrolment management with deletion.

Architecture tab — 3D graph. Logos: PyTorch/ExecuTorch, Core ML, Qualcomm, Android, Prometheus, Grafana. Strongest copy on **INT8 quantization**, **delegate selection**, **liveness**, **secure enclave storage**, and the **no-KV-cache** node explaining the substitution.

Monitoring tab — see below.

The Architecture tab must show the improvement loop. It is a stage card like any other, and it is the only one whose edges flow *back* upstream — that is the visual point. Give it the green treatment and its own tour stops; see §7 for the node inventory and 00-shared-requirements.md §2 for the graph mechanics (expandable pipeline, click-to-isolate, draggable nodes, canvas-drawn icon glyphs, constant-size labels, HTML detail panel).

The Monitoring tab has five sections, not four — Quality, Performance, Security, Audit and **Improvement**. The last one is never seeded with demo data.

7. Architecture graph: node inventory

These are the stage cards for this project’s Architecture tab, in pipeline order. Every one is a node; every child is a node inside its parent’s card. Build them against the ArchNode shape in 00-shared-requirements.md §8a, with all four rationale fields and a “Say this” line on each.

#	Stage card	Colour role	Expands into
0	Subject	neutral	— (the person being recognised, who has rights here)
1	Capture UI	teal	Camera preview, enrolment flow, consent capture, result display
2	Consent & Liveness Gate	amber	Explicit consent record, presentation-attack detection, on-device sandbox, no network path for imagery
3	Detect & Align	violet	Face detection, landmark alignment, quality scoring, rejection of unusable frames
4	ExecuTorch Runtime	orange	MobileFaceNet / ArcFace embedding model (INT8 PTQ) , XNNPACK / Core ML delegation, operator fusion, calibration set, “no KV cache — and why”
5	Matching	violet	Enrolment index, cosine similarity search, threshold policy, ambiguity refusal
6	Local Store	blue	Encrypted template store — templates only, never images , keystore-backed

#	Stage card	Colour role	Expands into
7	Monitoring & Audit	magenta	Biometric access log (hash-chained), consent records, erasure requests, opt-in aggregated metrics
8	Improvement Loop	green	False-match / false-reject reporting, threshold tuning from operator review, re-enrolment queue, promotion gate □ accuracy suite, on-device latency harness
9	Release Rings	grey	Signed model artifacts, staged OTA rings, canary devices, rollback

Two cards carry this project, and neither is accuracy. **Consent & Liveness Gate** is what makes the system deployable at all in a jurisdiction that treats faces as special-category data, and **Local Store** — templates, never images — is the single most reassuring sentence you can put in front of a buyer's legal team.

The no KV cache child under the runtime is deliberate. Keep it as a node with its detail panel explaining that a recognition model runs one forward pass per frame and has no autoregressive state to cache, and naming INT8 PTQ and delegate selection as the real optimisation. A client who knows the domain will notice a fabricated KV cache immediately.

8. The improvement loop for this project

00-shared-requirements.md §5a applies in full. What is specific here:

Biometrics invert the usual feedback economics: **the errors are the asset and the data is radioactive.**

- **False-reject reports** — the subject was enrolled and was not recognised. Capture the *embedding and the similarity score*, never the frame.
- **False-match reports** — someone was recognised as someone else. Rare, far more serious, and the one that must page a human.
- **Threshold review** — the operator adjusts the accept threshold and sees the projected effect on TAR@FAR from stored score distributions. This is the loop that actually improves the product day to day, and it needs no training run.

Hard rules, and they are not negotiable:

- **Never train on captured faces without separate, explicit, revocable consent for that specific purpose.** Consent to be recognised is not consent to become training data, and treating it as such is the failure mode that ends deployments.
- **Store score distributions, not imagery.** Threshold tuning needs the scores. It does not need the faces.
- **Erasure must remove the template, the scores and the enrolment.** Support GDPR Art. 17 as a working button, not a policy paragraph.

Most of the improvement here comes from threshold policy and enrolment quality, not from fine-tuning. Say that plainly rather than implying a training loop you would not actually be allowed to run.

8a. LoRA adapters: where they earn their place here

Be careful here — this is where LoRA is most often claimed and least often useful.

LoRA adapts a *training-time* graph. ExecuTorch consumes an exported, frozen, quantized graph. So the honest position is:

- **LoRA is a training technique in this project, not a serving one.** If you adapt the embedding backbone to a client's population, you do it with LoRA in PyTorch, then **merge the adapter into the base weights before export and quantization**. What ships to the device has no adapter in it.
- **There is no per-request adapter swapping on device.** Anything claiming otherwise for a mobile embedding model is describing a system that does not exist. Do not put an adapter-routing node in this architecture graph.

Where it is genuinely worth testing: domain adaptation of the embedding backbone when the deployment population differs markedly from the training distribution — a specific workforce, camera geometry or lighting environment. LoRA makes that adaptation cheap to try and cheap to abandon, which matters because most such attempts should be abandoned.

And a hard constraint that outranks the technique. Adapting a face model to a population means training on that population's faces, which requires separate, explicit, revocable consent for that specific purpose. Consent to be recognised is not consent to become training data. If that consent is not obtainable — and often it is not — the correct engineering answer is to improve enrolment quality and threshold policy instead, and to say so rather than quietly training anyway.

If you do adapt: re-run the full gate on the merged, re-quantized artifact, including the per-demographic breakdown. A backbone adapted to one population can lose accuracy on another, and an aggregate number will hide it.

9. The release gate for this project

00-shared-requirements.md §5b applies in full: two axes, either one able to block the release on its own.

Dimension	Question	Tool	Thresholds that matter here
Quality	Is it still right?	Held-out verification suite reporting TAR @ FAR = 1e-4 and 1e-6 , plus per-demographic breakdown	A single accuracy number is inadequate and, here, irresponsible. Report accuracy per demographic group ; a compression step that costs 2 points overall may cost 8 in one group, and shipping that is both a product failure and a legal one
Performance	Is it still fast and cool enough?	On-device harness: p95 end-to-end latency, peak memory, energy per recognition, sustained-throughput thermal behaviour	Recognition happens in a queue of real people. p95 above roughly 300 ms is felt as hesitation

1m_eval and GuideLLM do not apply — there is no language model and no serving endpoint. Substitute the two harnesses above and state the substitution explicitly, as section 4 of the shared requirements demands.

10. Monitoring and KPIs

Section	Metrics
Quality	TAR@FAR, false accept and false reject rates, broken out by demographic group , liveness detection rate, fallback-to-passcode rate

Section	Metrics
Performance	End-to-end verification latency p50/p95/p99, per-stage breakdown, model size, peak RAM, battery per verification, delegate in use, thermal events
Security	Spoof attempts detected, failed verifications per credential (credential-stuffing analogue), enclave access failures, model integrity failures
Audit	Every enrolment, verification, deletion: outcome, model version, threshold — never the image or template
Improvement	False-reject and false-match reports, score distribution drift, projected TAR@FAR at the current threshold, re-enrolment queue depth — per demographic group

The Improvement section is never seeded with demo data. Every other section falls back to synthetic numbers when the metrics backend is absent and says so with a badge. Feedback is a claim about what real people judged; a plausible approval rate nobody gave is the one number here that cannot be corrected by waiting for real traffic. An empty loop renders as empty.

The fairness breakdown belongs on the dashboard, not in a one-time report. Accuracy drift affects groups unevenly, and a system that was fair at launch can stop being fair as the deployed population changes. If it is not measured continuously it is not managed.

11. Security, audit and compliance

This project carries the heaviest compliance load in the pack. Face templates are **special-category personal data under GDPR Art. 9** and generally require explicit consent, with several jurisdictions imposing additional biometric-specific statutes.

- **Explicit, informed, revocable consent** captured at enrolment and logged. Not a checkbox in a EULA — a specific consent to biometric processing that states what is stored, where, and for how long.
- **Erasure (Art. 17) must be immediate and complete.** Deleting the template from the enclave is the entire deletion, because there is no other copy. That is a strong property — make it demonstrable in the demo.
- **Data minimisation:** store the template, never the image. A template is not reversible to a photograph in the way a stored image obviously is.
- **On-device processing means no international transfer** and no third-party processor for the biometric step. That is a genuinely simpler compliance posture and worth stating precisely.
- **Audit log** records verification events and outcomes, hash-chained, containing no biometric data itself.
- **Retention:** define it explicitly, including what happens when an employee leaves or a device is de-commissioned.

Do not claim certification. Show the controls and let the client's counsel assess them.

12. Deployment

Same shape as Project 03: signed .pte artifacts versioned independently of the app binary, staged OTA rollout rings with halt and rollback, TestFlight and Play testing tracks as the staging → production path.

Additional obligation here: a model update changes matching behaviour. A threshold that was correctly calibrated for the old embedding may be wrong for the new one, silently raising the false accept rate. Re-run the quality gate including the fairness evaluation before any rollout, and treat threshold recalibration as part of the model release, not a follow-up.

13. Deliverables

- Enrolment and verification flows, fully on-device

- Liveness detection with a working spoof demo
- ExecuTorch export pipeline with INT8 quantization and delegate selection
- Quality gate including **worst-group fairness gating**
- All three tabs
- Local hash-chained audit log with consent records
- Signed model artifacts with staged rollout
- Measured accuracy and latency across at least two device tiers

Acceptance: 1. Verification completes in under 500 ms p95 on a mid-range device 2. Printed photo and screen replay are both rejected by liveness 3. Fairness gate fails when a deliberately skewed model is supplied 4. Deleting an enrolment removes the template from the enclave, verifiably 5. No image or template appears in any log, packet capture or backup 6. Architecture tab and guided tour run offline

14. Out of scope

1:N identification against a gallery — explicitly excluded, see §1. Surveillance, covert capture, or any operation without the subject's active participation. Emotion or demographic inference from faces. Age estimation. Cross-device template sync, which would require a central store and undermine the architecture.

Project 05 — Workplace Safety Tracking (ONNX Runtime)

Read `00-shared-requirements.md` first. Everything in it applies here, with the device-class substitutions in §4.

0. Numbered build plan

Paste this section into Claude Code's plan mode to scope the work. Each step is independently reviewable; steps 1–4 must land before the demo is filmable.

1. **Repository skeleton.** Monorepo with `frontend/`, `backend/`, `model/`, `deploy/`, `observability/`. Strict TypeScript, typed Python, lint + types + tests in CI from the first commit.
2. **ONNX Runtime serving layer.** Stand up the engine with the compression spec in §3–§4 below. Record weights and latency before and after, and state what the freed resource buys in the units the client pays for.
3. **Backend API.** The component chain in §5 as real services: authentication, authorization, adversarial-input detection, context assembly, inference, guardrails, structured logging, hash-chained audit log.
4. **Product tab.** The working demo, usable by someone who has never seen it, plus the configuration surface. Configuration changes must visibly change behaviour and be inspectable before they apply.
5. **Architecture tab.** The 3D expandable-pipeline graph, built to the spec in `00-shared-requirements.md` §2. Use the node inventory in §7 of this document as the content. Click-to-isolate, draggable nodes, canvas-drawn icon glyphs, constant-size labels, HTML detail panel with all four rationale fields plus a "Say this" line.
6. **Guided tour.** An ordered walk covering every node that carries the commercial argument, including the honest stop about what is *not* enabled. Arrow keys, autoplay, and it must run with the backend stopped.
7. **Monitoring tab.** Quality, Performance, Security, Audit, Improvement. Live data when available, seeded demo data otherwise, honest source badge — except Improvement, which is never seeded.
8. **Feedback capture.** Rating, side-by-side comparison and correction in the Product tab, per §8. Redact on write, stamp the model/config version, store the triple denormalised, mark what an export consumed.
9. **Export and training path.** JSON Lines in the trainer's own format, resumable. Training is operator-run, never automatic. Write a training manifest next to every artifact. Evaluate **LoRA adapters** per §8a — that section says where they belong in *this* project and where they do not.
10. **Release gate.** Both axes per §9 — quality and performance, each able to fail the build on its own. Wire it into CI.
11. **Deployment.** Containers, infrastructure as code, staging → production with a rollback, and autoscaling on the signal that actually saturates.
12. **README.** A stranger can reach a working demo. Ends with the cost/performance/accuracy tradeoff table, one row per real decision, and a portability section.

Acceptance is §13 of this document plus the shared deliverables checklist.

1. Brief

Build an **edge safety monitor** for industrial floors and warehouses. Cameras watch a working area; the system detects people and vehicles, tracks them frame to frame, and raises an alert when someone enters a hazard zone or a forklift and a pedestrian converge.

The defining constraint:

Video never leaves the site. Inference runs on a device on the local network. The only things that travel are anonymous events — "person entered zone 3 at 14:22, dwell 4 s" — never frames, never identities.

That is both the privacy design and the economic one. Streaming twelve cameras to the cloud costs bandwidth continuously and creates a video archive somebody has to govern; processing locally and emitting events costs neither.

Design it as a safety system, not a surveillance system. It counts and tracks anonymous objects — no face recognition, no identification, no per-worker productivity metrics. That boundary is what makes

it deployable in workplaces with works councils or union agreements, and it should be enforced in the architecture, not just promised.

Target user: manufacturing, logistics, construction, ports — sites with vehicle and pedestrian mixing, PPE requirements and a genuine incident rate.

2. Why ONNX Runtime for this workload

- **One graph, many accelerators.** Edge deployments are heterogeneous by nature: some sites have a Jetson, some an Intel NUC, some an ARM box already installed. ONNX Runtime's **execution providers** — TensorRT, OpenVINO, CUDA, DirectML, CoreML, plain CPU — run the same .onnx file on all of them, choosing the fastest available at load time. Maintaining one model artifact across a mixed fleet is a large operational saving and the honest headline reason to choose it.
- **Mature INT8 quantization tooling** with both static (calibrated) and dynamic paths, plus per-channel weight quantization, which detection models need.
- **Graph optimisations** — constant folding, node fusion, layout transforms — applied at session creation with no code change.
- **It is a stable, boring, well-supported runtime.** For a device bolted to a wall in a warehouse for three years, that is a feature.

3. No KV cache here — the substitution

Per shared requirements §4, state this on the architecture node:

A detector runs **one forward pass per frame**. There is no autoregressive generation, no KV state, no KV cache, and no continuous batching in the LLM sense.

What replaces it, and what the demo must show instead:

Technique	What it buys
INT8 static quantization (calibrated on site footage)	~4× smaller, 2–4× faster; integer units are what edge NPUs are built for
Execution provider selection	TensorRT on Jetson, OpenVINO on Intel — same file
Graph fusion	Conv+BN+activation folded; fewer memory round trips
Input resolution	The largest latency lever in vision — and a direct recall trade on small/distant objects
Frame batching across cameras	Genuinely applicable here: batch N camera frames into one forward pass. This <i>is</i> the batching story, and it is real
Frame skipping + tracker interpolation	Detect at 10 FPS, track at 30 — the biggest single cost reduction available

That last one deserves emphasis in the client-facing copy: **the tracker is essentially free compared to the detector**, so running detection on every third frame and letting the tracker carry the intermediate frames cuts compute by roughly two thirds with little accuracy loss on slow-moving subjects. It is the highest leverage decision in the project and exactly the kind of engineering judgement a client is paying for.

4. Models and compression

Stage	Model	Notes
Detection	YOLOv8n / YOLO11n exported to ONNX	Classes: person, forklift, vehicle, and PPE items if required
Tracking	ByteTrack or OC-SORT	CPU, no learned model, cheap and robust
Optional	Pose model for fall detection	Only if the site's incident profile justifies it

Compression spec - INT8 static quantization calibrated on **footage from the actual site** — lighting, camera angle, motion blur and occlusion patterns differ enormously between a warehouse and a dockside, and calibration data from the wrong environment leaves accuracy on the table. - Record per execution provider: model size, FPS, mAP delta, power draw. - Ship an FP16 variant for comparison; on some accelerators FP16 is as fast as INT8 with less accuracy loss, and you cannot know without measuring.

Quality gate - mAP@50 and mAP@50-95 on a held-out set from the same site, gated as a delta against FP32. - **Recall on small and distant objects specifically.** This is where INT8 damage concentrates, and in a safety system a missed distant pedestrian is the failure that matters. Aggregate mAP hides it. - Tracking metrics: MOTA, IDF1, ID switches. A detector that is accurate per frame but swaps identities constantly produces useless dwell times and false alerts. - Sustained FPS on target hardware, measured over a long run, not a burst.

5. Architecture

Camera (RTSP) → Edge device on the local network

- Stream ingest, decode, frame buffer
- Device authentication (mTLS to the local hub)
- Authorization (which cameras, which zones)
- Frame sampling / skip policy
- ONNX Runtime detection
 - INT8 quantized .onnx
 - execution provider: TensorRT / OpenVINO / CPU
 - multi-camera frame batching
- Multi-object tracker (ByteTrack)
- Zone logic + rules engine ← dwell, intrusion, proximity, PPE
- Confidence + persistence gate ← N consecutive frames before alerting
- Anonymous event emission ← no frames, no identities
- Stateless structured logs
- Append-only audit log, hash-chained
- Local metrics ... sync ... Prometheus → Grafana
- Model registry → canary device → staged fleet rollout ← replaces Kubernetes
 - Fleet rollout (canary cameras, signed artifacts)

== IMPROVEMENT LOOP (the RLHF analogue) =====

- Missed detection → reviewer draws a box → that box is a label
- False detection → reviewer rejects a box
- ID switch → reviewer marks the frame (TRACKER signal, kept separate from detector feedback)
- store the FRAME with the correction, at model resolution
- Retraining set + class-level scoring → Grafana
- Retrain (operator-run, never automatic)
- Promotion gate: per-class mAP suite + sustained-FPS harness

==> back to the ONNX detector

Build deliberately:

- **The persistence gate is what makes it usable.** Alerting on a single frame's detection produces constant false alarms, and a safety system that cries wolf gets switched off within a week — which is the real failure mode of this product category. Require N consecutive frames, and make N configurable per rule.
- **Zones are drawn on the camera image, not typed as coordinates.** A site manager must be able to configure this without an engineer. Put the zone editor in the Product tab.
- **Events are anonymous by construction.** Track IDs are ephemeral and reset per session. They exist to link a person's frames within one event, not to follow anyone across time — and the architecture should make persistent tracking impossible rather than merely disabled.
- **Store nothing by default.** Optionally retain a short pre/post clip on an alert for incident review, with retention and access control stated. That is the one place video is kept, and it should be a deliberate,

visible exception.

6. Frontend

Product tab — live camera view with detection boxes, track IDs and trails, zone overlays, and an event feed. **A zone editor** — draw a polygon on the image, set the rule (no-entry, dwell limit, proximity threshold), set severity. An alert timeline.

Configuration: cameras, zones, rules, classes tracked, confidence and persistence thresholds, detection frame rate.

Architecture tab — 3D graph. Logos: ONNX, NVIDIA Jetson, Intel OpenVINO, Prometheus, Grafana. Strongest copy on **INT8 quantization, execution provider selection, frame skipping + tracker interpolation, the persistence gate**, and the **no-KV-cache** node explaining the substitution.

Monitoring tab — see below.

The Architecture tab must show the improvement loop. It is a stage card like any other, and it is the only one whose edges flow *back* upstream — that is the visual point. Give it the green treatment and its own tour stops; see §7 for the node inventory and `00-shared-requirements.md` §2 for the graph mechanics (expandable pipeline, click-to-isolate, draggable nodes, canvas-drawn icon glyphs, constant-size labels, HTML detail panel).

The Monitoring tab has five sections, not four — Quality, Performance, Security, Audit and **Improvement**. The last one is never seeded with demo data.

7. Architecture graph: node inventory

These are the stage cards for this project’s Architecture tab, in pipeline order. Every one is a node; every child is a node inside its parent’s card. Build them against the ArchNode shape in `00-shared-requirements.md` §8a, with all four rationale fields and a “Say this” line on each.

#	Stage card	Colour role	Expands into
0	Camera	neutral	— (the stream, and whatever it is pointed at)
1	Viewer Console	teal	Live overlay, zone editor, class selection, incident review
2	Ingest & Security	amber	RTSP/WebRTC ingest, TLS, authentication, authorization, stream-source validation, rate limiting
3	Frame Pipeline	violet	Decode, resize, letterbox, normalisation, frame skipping policy
4	ONNX Runtime Detector	orange	YOLO-class detector (INT8 static quantization) , graph fusion, execution provider selection, IO binding, dynamic batching, “no KV cache — and why”
5	Tracker	violet	Detection-to-track association, Kalman prediction, ID management, track lifecycle

#	Stage card	Colour role	Expands into
6	Rules & Zones	violet	Zone occupancy, dwell time, line crossing, alert policy, confidence gating
7	Database	blue	PostgreSQL, track and event store, model registry
8	Monitoring & Audit	magenta	Prometheus, Grafana, hash-chained audit log, retention policy, structured logging
9	Improvement Loop	green	Missed/false detection reporting from review, corrected-frame queue, class-level scoring, retraining set, promotion gate □ mAP suite, FPS harness
10	Fleet	grey	Edge device fleet, canary cameras, signed model artifacts, staged rollout, rollback

The card carrying the argument is **ONNX Runtime Detector**, and the honest framing is throughput per pound rather than raw accuracy: INT8 plus graph fusion plus the right execution provider is the difference between one box handling four cameras and one box handling sixteen. That ratio is the number the buyer is actually purchasing.

ID switches, not mAP, are what users notice. A tracker that swaps two people's identities as they cross produces a report nobody trusts, however good the detector's mAP is. Give the tracker its own strong detail-panel copy.

8. The improvement loop for this project

00-shared-requirements.md §5a applies in full. What is specific here:

The feedback here is **spatial**, which changes how it must be captured:

- **Missed detection** — the reviewer draws a box on a frame where nothing was detected. That box is a training label.
- **False detection** — the reviewer rejects a box. Equally valuable, and much more common in practice.
- **ID switch** — the reviewer marks the frame where two tracks swapped. This is the tracker's signal, not the detector's, and conflating them means tuning the wrong component.

Rules for this domain:

- **Store the frame with the correction**, at the resolution the model saw. A label without its pixels is not a training example, and re-decoding the source video later is unreliable once retention has run.
- **Separate detector feedback from tracker feedback.** They train different things; a queue that mixes them teaches neither.
- **Sample the boring frames too.** A dataset built only from frames someone bothered to correct is biased toward hard cases, and a model trained on it becomes over-eager on easy ones.
- **Retention and faces.** If the frames contain identifiable people, the correction queue is personal data. Blur non-target regions on write, or set a short retention and say what it is.

8a. LoRA adapters: where they earn their place here

Test this, but scope it honestly.

ONNX Runtime executes a frozen, fused, quantized graph. There is no adapter swapping at inference time, and no per-request adapter routing. LoRA belongs to the training half of this project:

- Adapt the detector backbone with LoRA in PyTorch, **merge, then export to ONNX and quantize**. What runs on the edge box is a single fused graph.
- Do not draw an adapter node in the serving path of the architecture graph. Draw it in the improvement loop, where it belongs.

Where it genuinely earns its place: per-site adaptation. Detector accuracy on a fixed camera is dominated by that camera’s specifics — mounting angle, lens, lighting, what the floor looks like, what people wear. A model that is excellent on a public benchmark is frequently mediocre on one particular loading bay, and generic retraining to fix one site risks the other forty.

LoRA makes per-site adaptation affordable to try:

Approach	Cost to adapt one site	Risk to other sites
Retrain the whole detector	high	high — a global change for a local problem
LoRA on the backbone, merged per site	low	contained: one site’s artifact

The fleet already ships signed per-device artifacts, so a site-specific model is a rollout target you have anyway.

Measure: per-class mAP on that site’s held-out frames *and* on the general benchmark, before and after. An adaptation that gains 6 points locally and loses 4 globally is usually the wrong trade, and only measuring locally hides it completely.

9. The release gate for this project

00-shared-requirements.md §5b applies in full: two axes, either one able to block the release on its own.

Dimension	Question	Tool	Thresholds that matter here
Quality	Is it still right?	Held-out detection suite: mAP@50 and mAP@50-95 per class , plus tracking metrics (MOTA, IDF1, ID switches)	Gate per class , not on the aggregate. A quantization step that costs 1 point of mAP overall while destroying the smallest and most important class is a regression the aggregate hides
Performance	Is it still fast enough for real time?	FPS harness on the target hardware at the real input resolution, plus p95 per-frame latency and memory	Sustained FPS at the deployed resolution is an absolute floor. Below real time the tracker starts missing associations and ID switches climb — the quality metric degrades because the performance metric did

lm_eval and GuideLLM do not apply. Substitute the two harnesses above, benchmark at the **deployed resolution and frame rate**, and run long enough for thermal throttling to appear on edge hardware.

10. Monitoring and KPIs

Section	Metrics
Quality	mAP, recall on small objects, ID switches per hour, false alert rate per camera per shift , missed-event rate against a labelled audit sample
Performance	FPS per camera, detection latency p50/p95/p99, end-to-end alert latency, GPU/NPU utilisation, device temperature , dropped frames
Security	Camera stream authentication failures, unauthorised zone config changes, model integrity failures, tampering (camera moved or obscured)
Audit	Every alert, every config change, every clip retained or accessed — hash-chained
Improvement	Missed/false detections reported per class, ID-switch reports, corrected frames awaiting retraining, class-level accuracy trend

The Improvement section is never seeded with demo data. Every other section falls back to synthetic numbers when the metrics backend is absent and says so with a badge. Feedback is a claim about what real people judged; a plausible approval rate nobody gave is the one number here that cannot be corrected by waiting for real traffic. An empty loop renders as empty.

False alert rate per shift is the metric the client actually cares about, and it should be the headline. Detection mAP is an engineering number; “your team got four false alarms last week instead of forty” is the business case.

Dropped frames and device temperature are early warning signs. An edge box in a warehouse ceiling will thermally throttle in summer, and the first symptom is dropped frames, not an outage. Alert on it before it becomes a missed incident.

11. Security, audit and compliance

- **Workplace monitoring has specific legal obligations** in many jurisdictions — works council consultation in Germany, employee notification requirements elsewhere. The system must support a **transparency report**: what is detected, what is stored, for how long, who can see it. Build it as a product feature.
- **GDPR**: even anonymous-by-design tracking can constitute personal data processing if individuals are identifiable in context. Legitimate interest is the usual basis for safety monitoring; data minimisation is served by emitting events rather than video. Document the DPIA inputs.
- **No biometric identification.** Enforced architecturally — no face model is present in the pipeline. That is a strong statement to a works council, and it is only credible if it is structurally true.
- **Camera streams authenticated with mTLS.** An unauthenticated RTSP stream on a factory network is an obvious entry point.
- **Signed model artifacts**, verified before load.
- **Retention**: events long, clips short, frames never.

12. Deployment

Edge fleet, not Kubernetes — though K3s on the edge boxes is a reasonable way to get declarative deployment and rollback, and worth considering.

- **Canary device □ site □ fleet** rollout rings. A bad model reaching every camera at every site simultaneously is the failure this structure prevents.
- Signed, versioned model artifacts fetched from a registry.
- **Devices must operate through network loss.** A safety alert cannot depend on a cloud round trip. Buffer events locally and sync when the link returns; alerting is local and immediate.
- Remote health check, log retrieval and rollback without a site visit — the operational cost of physically visiting devices dominates everything else at fleet scale.

13. Deliverables

- Working edge pipeline: RTSP in, tracked detections and alerts out
- Zone editor usable by a non-engineer
- ONNX export and INT8 quantization pipeline with a quality gate
- Execution provider benchmark table across at least two accelerators
- All three tabs
- Hash-chained audit log
- Fleet rollout with canary and rollback
- Measured FPS, latency and false alert rate on real footage

Acceptance: 1. Sustained 30 FPS effective tracking on the target device with the stated camera count, measured over an hour, not a burst 2. Quality gate fails on a model with degraded small-object recall 3. Alert fires within 2 s of a zone intrusion 4. No frames leave the device except explicitly retained incident clips 5. Device survives simulated network loss without missing local alerts 6. Architecture tab and guided tour run with the device offline

14. Out of scope

Face recognition or any identification of individuals — explicitly excluded. Productivity or behaviour monitoring of named workers. Re-identification across cameras or across sessions. Cloud video archive. PTZ camera control. Training a custom detector from scratch; start from pretrained weights and fine-tune if needed.

Project 06 — Drone Infrastructure Inspection (LiteRT)

Read `00-shared-requirements.md` first. Everything in it applies here, with the device-class substitutions in §4.

0. Numbered build plan

Paste this section into Claude Code's plan mode to scope the work. Each step is independently reviewable; steps 1–4 must land before the demo is filmable.

1. **Repository skeleton.** Monorepo with `frontend/`, `backend/`, `model/`, `deploy/`, `observability/`. Strict TypeScript, typed Python, lint + types + tests in CI from the first commit.
2. **LiteRT serving layer.** Stand up the engine with the compression spec in §3–§4 below. Record weights and latency before and after, and state what the freed resource buys in the units the client pays for.
3. **Backend API.** The component chain in §5 as real services: authentication, authorization, adversarial-input detection, context assembly, inference, guardrails, structured logging, hash-chained audit log.
4. **Product tab.** The working demo, usable by someone who has never seen it, plus the configuration surface. Configuration changes must visibly change behaviour and be inspectable before they apply.
5. **Architecture tab.** The 3D expandable-pipeline graph, built to the spec in `00-shared-requirements.md` §2. Use the node inventory in §7 of this document as the content. Click-to-isolate, draggable nodes, canvas-drawn icon glyphs, constant-size labels, HTML detail panel with all four rationale fields plus a “Say this” line.
6. **Guided tour.** An ordered walk covering every node that carries the commercial argument, including the honest stop about what is *not* enabled. Arrow keys, autoplay, and it must run with the backend stopped.
7. **Monitoring tab.** Quality, Performance, Security, Audit, Improvement. Live data when available, seeded demo data otherwise, honest source badge — except Improvement, which is never seeded.
8. **Feedback capture.** Rating, side-by-side comparison and correction in the Product tab, per §8. Redact on write, stamp the model/config version, store the triple denormalised, mark what an export consumed.
9. **Export and training path.** JSON Lines in the trainer's own format, resumable. Training is operator-run, never automatic. Write a training manifest next to every artifact. Evaluate **LoRA adapters** per §8a — that section says where they belong in *this* project and where they do not.
10. **Release gate.** Both axes per §9 — quality and performance, each able to fail the build on its own. Wire it into CI.
11. **Deployment.** Containers, infrastructure as code, staging → production with a rollback, and autoscaling on the signal that actually saturates.
12. **README.** A stranger can reach a working demo. Ends with the cost/performance/accuracy tradeoff table, one row per real decision, and a portability section.

Acceptance is §13 of this document plus the shared deliverables checklist.

1. Brief

Build an **onboard aerial inspection system** for infrastructure: solar farms, transmission lines, roofs, wind turbine blades. A drone flies a survey pattern; a companion computer analyses each frame in flight, flags defects — cracked or soiled panels, thermal hotspots, corrosion, vegetation encroaching on a line — geotags them, and produces a prioritised report before the aircraft has landed.

The constraint that makes this project unlike the others in the pack:

Compute costs flight time. Every watt the inference board draws is a watt not driving the rotors. On a 25-minute airframe, a 10 W processing load can cost two to three minutes of endurance — which is several hundred panels not surveyed per flight.

Nowhere else in this pack does model efficiency convert so directly into a business number. Optimisation here is not about cloud spend; it is about how much asset gets inspected per battery. That framing should run through the entire client-facing narrative.

Target user: utility-scale solar operators, transmission network owners, industrial roof surveyors, wind operators — anyone currently paying for manual inspection or reviewing thousands of aerial photos by hand.

2. Why LiteRT for this workload

LiteRT (formerly TensorFlow Lite) is built for exactly this envelope:

- **Full-integer INT8 quantization** producing models that run on EdgeTPU-class accelerators. A Coral EdgeTPU draws about 2 W and delivers several TOPS — the best inference-per-watt available in this form factor, and on a drone that ratio is the whole engineering problem.
- **Delegate ecosystem** — EdgeTPU, GPU, NNAPI, XNNPACK — so the same `.tflite` runs on a Coral, a Pi GPU, or bare ARM CPU with a graceful fallback. Useful because airframe payload options change and the software should not.
- **Small runtime.** The interpreter is a few hundred kilobytes, which matters on a constrained companion computer.
- **Model Optimization Toolkit** gives quantization-aware training, **pruning** and weight clustering in one pipeline. This is the one project in the pack where pruning genuinely earns its place, because the compute budget is hard-capped by physics rather than by cost.

The honest constraint: full-integer quantization for EdgeTPU is strict. Unsupported operators fall back to CPU and destroy the latency budget silently. Verify the compiled model has no CPU fallback segments, and make that a build-time check rather than something discovered in flight.

3. No KV cache here — the substitution

Per shared requirements §4:

A defect detector runs **one forward pass per captured frame**. There is no token generation, no KV state, no KV cache, and no continuous batching in the LLM sense.

What is demonstrated instead — and this project has the richest substitution story in the pack because the power budget forces every lever to be used:

Technique	What it buys
Full-integer INT8 quantization (QAT)	Required for EdgeTPU; ~4× smaller, order-of-magnitude better inference/watt
Structured pruning	Fewer FLOPs and less memory traffic — directly fewer joules per frame
Weight clustering	Smaller artifact, faster OTA updates over a field link
Delegate selection	EdgeTPU where fitted, GPU or XNNPACK fallback
Tiled inference at native resolution	Aerial defects are small; downscaling a 20 MP frame destroys them. Tile instead, and only tile where a coarse pass found something
Two-tier triage	Cheap onboard screening, expensive analysis on the ground
Adaptive capture rate	Slow the frame rate over uniform terrain, raise it over structures

The two-tier design is the key architectural idea. Onboard, run a small, fast model that answers “is there anything here worth keeping?” — discarding the 90%+ of frames showing intact panels. Store and later process only the candidates on a ground station with a larger model. This cuts onboard compute, onboard storage and post-flight review time simultaneously, and it is the decision a client should understand from the architecture graph.

4. Models and compression

Stage	Model	Where
Onboard triage	MobileNetV3-Small or EfficientDet-Lite0, INT8	Companion computer
Onboard detection	EfficientDet-Lite1 / YOLO-nano, INT8	Companion computer

Stage	Model	Where
Ground analysis	Larger segmentation model for defect extent	Ground station / laptop
Thermal (if fitted)	Small CNN on radiometric data	Onboard

Compression spec - Quantization-aware training, not post-training quantization, for the onboard models. Aerial defects are small, low-contrast features; PTQ tends to lose exactly those while leaving aggregate metrics looking acceptable. - Apply structured pruning, then QAT, then compile for EdgeTPU. Record at each step: model size, FLOPs, inference latency, **watts**, and accuracy. - **Verify zero CPU fallback** in the compiled EdgeTPU model, as a build gate.

Quality gate - Defect detection recall at a fixed precision, gated against the FP32 baseline. - **Recall on small defects specifically** — a hairline crack at 40 m altitude is a handful of pixels, and it is the whole point of the product. Aggregate mAP will not show its loss. - **Inference energy per frame** as a gated metric. If a model change costs flight time it must fail the build, exactly as an accuracy regression would. This is the gate that makes the project's core constraint enforceable rather than aspirational. - False positive rate: an inspection that flags every panel is worthless, because a human still reviews everything.

5. Architecture

Camera / thermal sensor on airframe

- Companion computer (Raspberry Pi / Jetson Orin Nano + Coral)
- Device authentication (signed boot, mTLS to ground station)
- Authorization (which survey, which asset)
- Frame capture + adaptive rate control
- LiteRT onboard triage (INT8, EdgeTPU) ← discard uninteresting frames
- LiteRT onboard detection on candidates
- Geotag from flight controller (MAVLink): GPS, altitude, gimbal attitude
- Confidence gate + defect classification
- Local storage: candidates only, with metadata
- Local append-only audit log, hash-chained
 - ... landing / link restored ...
- Ground station: larger model, orthomosaic, defect extent, report
- Prometheus → Grafana (fleet health, model performance)
- Model registry → canary aircraft → fleet rollout ← replaces Kubernetes
 - Fleet OTA (canary aircraft, signed artifacts)

== IMPROVEMENT LOOP (the RLHF analogue) ==

- Confirm / reject a finding – the inspector does this anyway
 Severity re-grading – model said minor, inspector said urgent
 Missed defect – added on review, rare and expensive
- store the TILE the model saw, with its geo-reference
 - record WHICH inspector: severity is expert judgement
 - Retrain (operator-run, never automatic)
 - Promotion gate: per-class IoU + recall-on-severe (absolute floor) + energy-per-frame harness

==> back to the LiteRT model

Build deliberately:

- **Geotagging accuracy is the deliverable.** “There is a defect” is not actionable; “there is a defect on panel row 14, string 7, at these coordinates” sends a technician to the right place. Fuse detection with GPS, altitude and gimbal attitude, and validate the projection against known ground points. A detection with a bad position is worse than no detection because it wastes a site visit.
- **The aircraft flies without a link.** All onboard decisions are local. Telemetry and results sync opportunistically; nothing waits on connectivity.

- **Storage is bounded and must degrade gracefully.** When the card fills mid-survey, drop lowest-confidence candidates first rather than stopping capture.
- **Never take flight-critical action.** This is an inspection payload, strictly advisory. It does not steer, does not trigger avoidance, and has no authority over the flight controller. Say this explicitly in the architecture graph — it is the first question any aviation-literate client asks, and the boundary must be structural.

6. Frontend

Product tab — a survey view: map with the flight path, defect pins coloured by severity, and a click-through to the source frame with the detection overlay. A prioritised defect list exportable as an inspection report. **A live in-flight view** showing frames analysed, candidates kept, and — prominently — **watts drawn and estimated flight time remaining**.

That power readout is this project’s equivalent of the KV-cache gauge in the vLLM project: the live, visible manifestation of the core engineering constraint, and the thing to point at on camera.

Configuration: asset type and defect classes, confidence thresholds, capture rate policy, triage aggressiveness with the compute/recall trade shown as a curve.

Architecture tab — 3D graph. Logos: LiteRT/TensorFlow, Coral EdgeTPU, Raspberry Pi, MAVLink/PX4, Prometheus, Grafana. Strongest copy on **INT8 quantization, pruning, two-tier triage, EdgeTPU delegation, adaptive capture**, and the **no-KV-cache** node explaining the substitution.

Monitoring tab — see below.

The Architecture tab must show the improvement loop. It is a stage card like any other, and it is the only one whose edges flow *back* upstream — that is the visual point. Give it the green treatment and its own tour stops; see §7 for the node inventory and `00-shared-requirements.md` §2 for the graph mechanics (expandable pipeline, click-to-isolate, draggable nodes, canvas-drawn icon glyphs, constant-size labels, HTML detail panel).

The Monitoring tab has five sections, not four — Quality, Performance, Security, Audit and **Improvement**. The last one is never seeded with demo data.

7. Architecture graph: node inventory

These are the stage cards for this project’s Architecture tab, in pipeline order. Every one is a node; every child is a node inside its parent’s card. Build them against the `ArchNode` shape in `00-shared-requirements.md` §8a, with all four rationale fields and a “Say this” line on each.

#	Stage card	Colour role	Expands into
0	Flight	neutral	— (the mission, and the asset being inspected)
1	Ground Console	teal	Mission planner, imagery review, finding triage, export
2	Ingest & Security	amber	Link encryption, device identity, authentication, authorization, artifact signature verification
3	Capture Pipeline	violet	Frame capture, exposure and blur rejection, tiling for high-resolution imagery, normalisation

#	Stage card	Colour role	Expands into
4	LiteRT Runtime	orange	Segmentation / defect model (INT8 full-integer) , NPU / EdgeTPU delegation, structured pruning, delegate fallback path, “no KV cache — and why”
5	Post-processing	violet	NMS, mask assembly, tile stitching, confidence gating, severity classification
6	Geo-referencing	violet	GPS/IMU fusion, tile-to-world mapping, finding localisation
7	Store & Sync	blue	On-board store, deferred sync when the link returns, finding registry
8	Monitoring & Audit	magenta	Prometheus, Grafana, hash-chained audit log, flight records, retention policy
9	Improvement Loop	green	Inspector confirmation and rejection of findings, corrected tiles, severity re-grading, retraining set, promotion gate □ segmentation suite, on-device latency and energy harness
10	Fleet OTA	grey	Signed model artifacts, canary aircraft, staged rollout rings, rollback

Two cards carry this project. **LiteRT Runtime**, where the delegate fallback path deserves its own strong copy — an NPU that silently falls back to CPU turns a 30 ms inference into 400 ms and drains the battery mid-flight, and a system that does not detect and report that failure will fail quietly in the field. And **Store & Sync**: an inspection drone works where there is no link, so “deferred sync” is not a nicety, it is the operating mode.

Energy is a first-class metric here in a way it is not anywhere else in this pack: inference that costs battery costs flight time, and flight time is the thing the client is buying.

8. The improvement loop for this project

00-shared-requirements.md §5a applies in full. What is specific here:

Inspection has a naturally high-quality feedback signal, because **a human already reviews every finding before it becomes a work order**:

- **Confirm / reject a finding** — the inspector is doing this anyway. Capture it. Zero extra effort, and it is a labelled example.
- **Severity re-grading** — the model said minor, the inspector said urgent. The most valuable signal in the project, because severity drives cost.
- **Missed defect** — added by the inspector on review. Rare, expensive, and the one worth paging about.

Rules for this domain:

- **Store the tile, not the whole frame.** High-resolution imagery is tiled for inference; the training example is the tile the model actually saw, at its actual resolution.
- **Keep the geo-reference with the correction.** A defect label without its location cannot be checked against the asset later, and cross-flight consistency (“did we see this crack last quarter?”) is a large part of the product’s value.
- **Weight by inspector, and record who.** Severity grading is expert judgement and experts disagree; a training set that averages a novice and a twenty-year inspector without distinguishing them loses the expertise.
- **Sync the queue opportunistically,** like everything else on the aircraft. The loop must tolerate days offline.

8a. LoRA adapters: where they earn their place here

Test this — the tiered-model story here makes LoRA unusually attractive.

As with the other vision projects, LiteRT runs a frozen, INT8, delegate-mapped graph: LoRA is a training-time technique, merged before conversion, with no adapter swapping in flight. Keep it in the improvement loop in the architecture graph, never in the onboard serving path.

Where it earns its place: per-asset-class adapters. Solar panels, wind turbine blades, transmission towers, roofs and bridges are different visual problems that share most of a backbone. One model that handles all of them is worse at each than five specialised models — but five full models is five training pipelines, five validation sets and five artifacts to keep in sync.

Approach	Training cost	Accuracy per asset class	Artifacts to manage
One general model	1×	worst	1
Full model per asset class	5×	best	5 independent
Shared backbone + LoRA per class, merged	~1.2×	close to best	5, but one lineage

The onboard model is chosen per mission — the flight plan already knows the asset class — so selection costs nothing at inference time.

The second case: seasonal and regional drift. A defect model tuned on summer imagery degrades on winter imagery: different sun angle, snow, different vegetation. LoRA makes a seasonal re-adaptation cheap enough to actually do each year rather than propose and defer.

Measure: per-class IoU and recall-on-severe for the adapted artifact against the general one, **and energy per frame after merging and re-quantizing.** A merged adapter should not change inference cost — verify that rather than assume it, because a change that quietly moves an operator off the NPU fallback path costs flight time.

9. The release gate for this project

00-shared-requirements.md §5b applies in full: two axes, either one able to block the release on its own.

Dimension	Question	Tool	Thresholds that matter here
Quality	Is it still right?	Held-out inspection suite: per-class IoU / Dice, and recall at the operating threshold , broken down by defect severity	Recall on severe defects is an absolute floor, not a delta. A missed critical defect is the failure that ends the contract; a false positive costs an inspector five minutes. Gate them asymmetrically and say why
Performance	Is it still fast and cheap enough to fly?	On-device harness: p95 inference latency, energy per frame , sustained throughput with thermal soak, and delegate-fallback rate	Energy is flight time. A model that is 10% more accurate and 40% more expensive per frame may be a worse product, and the gate should make that visible rather than let accuracy win by default

`lm_eval` and `GuideLLM` do not apply. Substitute the harnesses above, measure on the actual airframe compute with the actual thermal envelope, and **fail the gate if the NPU delegate falls back to CPU at all** — a silent fallback is the single most likely way this system degrades in production.

10. Monitoring and KPIs

Section	Metrics
Quality	Defect recall and precision by defect class and size, false positive rate, geotag positional error, ground-truth agreement on audited samples
Performance	Frames/sec onboard, inference latency, watts and joules per frame, flight minutes consumed by compute , triage discard rate, thermal throttling, storage headroom
Security	Signed boot failures, model integrity failures, unauthorised survey access, link authentication failures
Audit	Every survey: aircraft, operator, model version, area covered, defects found, chain of custody for the report
Improvement	Findings confirmed vs rejected, severity re-grades (by inspector), missed defects reported, tiles awaiting retraining

The Improvement section is never seeded with demo data. Every other section falls back to synthetic numbers when the metrics backend is absent and says so with a badge. Feedback is a claim about what real people judged; a plausible approval rate nobody gave is the one number here that cannot be corrected by waiting for real traffic. An empty loop renders as empty.

The headline business metric: assets inspected per battery, next to the manual inspection baseline. Everything technical should be traceable to it. A 20% reduction in inference power becomes a directly quotable increase in panels surveyed per flight, and that is the number that closes the sale.

Audit chain of custody matters here more than it might appear: inspection reports feed warranty claims and regulatory compliance. Which model version produced a finding, and whether it can be reproduced, is

a question that will be asked when a manufacturer disputes a claim.

11. Security, audit and compliance

- **Aviation regulation is the primary constraint.** The payload must be advisory only, with no authority over flight. Document that boundary explicitly; operators will be asked to demonstrate it.
- **Signed boot and signed model artifacts.** An aircraft is physically accessible and periodically unattended.
- **Imagery may capture people and property incidentally.** Even inspecting a solar farm, frames will contain vehicles, roads and occasionally people. Minimise: discard non-candidate frames onboard, and blur or discard incidental subjects in retained frames. State the retention policy.
- **Airspace and privacy law vary sharply by jurisdiction.** The system should record the survey boundary and flag captures outside it rather than assuming compliance.
- **Chain of custody** for inspection reports: hash-chained audit log linking each finding to aircraft, flight, model version and operator.

12. Deployment

- **Canary aircraft** □ **squadron** □ **fleet** rollout rings for model updates. A bad model that under-detects reaching an entire fleet means a season of surveys that must be repeated.
- Signed, versioned `.tf` artifacts; small artifacts matter because updates go over field links.
- **Rollback must work without a laptop on site.** Field crews are not engineers.
- Ground station deployable as a container so a site can run it on a laptop disconnected from anything.

13. Deliverables

- Onboard pipeline: capture □ triage □ detect □ geotag □ store
- Ground station: larger-model analysis, map view, inspection report export
- LiteRT pipeline with pruning, QAT, EdgeTPU compilation, and a **no-CPU-fallback** build check
- Quality gate including small-defect recall **and energy per frame**
- All three tabs
- Hash-chained audit log with chain of custody
- Fleet rollout with canary and rollback
- Measured watts, flight-time cost, and detection accuracy on real survey footage

Acceptance: 1. Onboard pipeline sustains the target capture rate within the stated power budget, measured with a real power meter, not a datasheet figure 2. Compiled EdgeTPU model contains zero CPU fallback segments 3. Quality gate fails a model that regresses small-defect recall **or** exceeds the energy budget 4. Geotag positional error within the agreed tolerance against surveyed ground points 5. Full survey completes with no network link 6. Architecture tab and guided tour run offline on the ground station

14. Out of scope

Flight control, autonomy, obstacle avoidance — advisory payload only. Person detection or tracking; incidental subjects are minimised, not analysed. Beyond visual line of sight operations, which carry their own certification burden. Photogrammetry and 3D reconstruction beyond a simple orthomosaic. Real-time video downlink.